The Concise TypeScript Book

Simone Poggiali

The Concise TypeScript Book

『The Concise TypeScript Book』は、TypeScript の機能を包括的かつ簡潔に解説します。強力な型システムから高度な機能まで、最新バージョンの言語に関するあらゆる側面を明快に説明します。

初心者にも経験豊富な開発者にも、本書は TypeScript への理解を深め、習熟度を高めるうえで非常に価値のある資料です。

本書は完全に無料で、オープンソースです。

質の高い技術教育は誰もが利用できるべきだと私は考えています。そのため、本書を無料で公開し続け、改善や新しい例を加えて定期的に更新しています。

The Concise TypeScript Book Plus Edition をご覧ください。



オープンソース版の先へ進みたい読者に向けて、**The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** には、実践的な活用に焦点を当てた追加の限定コンテンツが収録されています。

Plus Edition には以下が含まれます。

- **TypeScript 7 対応** — TypeScript 7 の最新機能と言語の改善点を解説します。
- **React で使う TypeScript** — コンポーネント、props、フック、イベント、children、ref、および一般的な React パターンの型付けについて実践的に解説します。

- **実際のプロジェクトのための TypeScript レシピ** — TypeScript アプリケーションの構築と保守で開発者が直面する実践的な問題を扱う、焦点を絞った例を紹介します。

Plus Edition をご購入いただくことで、無料のオープンソース版の継続的な開発と保守を直接支援することにもなります。

Plus Edition は、世界各国の Amazon で英語版とイタリア語版を入手できます。Plus Edition の詳細を確認し、Amazon で購入する。

プロジェクトを支援する

無料版がバグの修正、難しい概念の理解、キャリアの前進に役立ったなら、希望する金額（推奨額は **$5**）をお支払いいただくか、コーヒー 1 杯分のご支援をご検討ください。

皆さまのご支援により、コンテンツを最新の状態に保ち、新しい例、より明快な説明、実践的なガイダンスを追加して充実させることができます。

BUY ME A COFFEE

Donate PayPal

翻訳

本書は、以下を含む複数の言語に翻訳されています。

英語

ブルガリア語

ドイツ語

フランス語

インドネシア語

イタリア語

日本語

韓国語

ポーランド語

ポルトガル語（ブラジル）

スウェーデン語

トルコ語

ベトナム語

中国語

スペイン語

タイ語

ロシア語

アラビア語 チェコ語

ダウンロードとウェブサイト

EPUB 版もダウンロードできます。

https://github.com/gibbok/typescript-book/tree/main/downloads

オンライン版は以下で利用できます。

https://gibbok.github.io/typescript-book

目次

- The Concise TypeScript Book
  - プロジェクトを支援する
  - 翻訳
  - ダウンロードとウェブサイト
  - 目次
  - はじめに
  - 著者について
  - TypeScript 入門
    - TypeScript とは？
    - TypeScript を選ぶ理由
    - TypeScript と JavaScript
    - TypeScript のコード生成
    - モダン JavaScript を今すぐ使う（ダウンレベリング）
  - TypeScript を始める
    - インストール
    - 構成
    - TypeScript 構成ファイル
      - target
      - lib

        - strict
        - module
        - moduleResolution
        - esModuleInterop
        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - TypeScript への移行に関するアドバイス
- 型システムを探る
    - TypeScript Language Service
    - 構造的型付け
    - TypeScript の基本的な比較規則
    - 集合としての型
    - 型を割り当てる：型宣言と型アサーション
        - 型宣言
        - 型アサーション
        - アンビエント宣言
    - プロパティチェックと過剰プロパティチェック
    - 弱い型
    - 厳密なオブジェクトリテラルチェック（Freshness）
    - 型推論
    - より高度な推論
    - 型の拡大
    - const
        - 型パラメーターの const 修飾子
        - const アサーション

        - 明示的な型アノテーション
        - 型の絞り込み
            - 条件
            - throw または return
            - 判別可能なユニオン
            - ユーザー定義型ガード
            - switch-true による絞り込み
    - プリミティブ型
        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null と undefined
        - Array
        - any
    - 型アノテーション
    - オプショナルプロパティ
    - 読み取り専用プロパティ
    - インデックスシグネチャ
    - 型の拡張
    - リテラル型
    - リテラル推論
    - strictNullChecks
    - 列挙型
        - 数値列挙型
        - 文字列列挙型
        - 定数列挙型
        - 逆引きマッピング

        - アンビエント列挙型
        - 計算メンバーと定数メンバー
    - 型の絞り込み
        - typeof 型ガード
        - 真偽性による絞り込み
        - 等価性による絞り込み
        - in 演算子による絞り込み
        - instanceof による絞り込み
    - 代入
    - 制御フロー解析
    - 型述語
    - 判別可能なユニオン
    - never 型について
    - 網羅性チェック
    - オブジェクト型
    - タプル型（匿名）
    - 名前付きタプル型（ラベル付き）
    - 固定長タプル
    - ユニオン型
    - 交差型
    - 型のインデックスアクセス
    - 値から型を取得する
    - 関数の戻り値から型を取得する
    - モジュールから型を取得する
    - マップ型
    - マップ型の修飾子
    - 条件型
    - 分配条件型
    - 条件型における infer 型推論

- 定義済み条件型
- テンプレートユニオン型
- any 型
- unknown 型
- void 型
- never 型
- インターフェースと型
  - 共通の構文
  - 基本型
  - オブジェクトとインターフェース
  - ユニオン型と交差型
- 組み込みのプリミティブ型
- 一般的な組み込み JavaScript オブジェクト
- オーバーロード
- マージと拡張
- 型とインターフェースの違い
- クラス
  - クラスの一般的な構文
  - コンストラクター
  - プライベートコンストラクターとプロテクテッドコンストラクター
  - アクセス修飾子
  - ゲッターとセッター
  - クラスの自動アクセサー
  - this
  - パラメータープロパティ
  - 抽象クラス
  - ジェネリクスを使用するクラス
  - デコレーター

        - クラスデコレーター
        - プロパティデコレーター
        - メソッドデコレーター
        - ゲッターとセッターのデコレーター
        - デコレーターメタデータ
    - 継承
    - 静的メンバー
    - プロパティの初期化
    - メソッドのオーバーロード
- ジェネリクス
    - ジェネリック型
    - ジェネリッククラス
    - ジェネリック制約
    - ジェネリクスのコンテキストに基づく絞り込み
- 消去される構造的型
- 名前空間
- シンボル
- トリプルスラッシュディレクティブ
- 型の操作
    - 型から型を作成する
    - インデックスアクセス型
    - ユーティリティ型
        - Awaited<T>
        - Partial<T>
        - Required<T>
        - Readonly<T>
        - Record<K, T>
        - Pick<T, K>
        - Omit<T, K>

        - Exclude<T, U>
        - Extract<T, U>
        - NonNullable<T>
        - Parameters<T>
        - ConstructorParameters<T>
        - ReturnType<T>
        - InstanceType<T>
        - ThisParameterType<T>
        - OmitThisParameter<T>
        - ThisType<T>
        - Uppercase<T>
        - Lowercase<T>
        - Capitalize<T>
        - Uncapitalize<T>
        - NoInfer<T>
    - その他
        - エラーと例外処理
        - ミックスインクラス
        - 非同期言語機能
        - イテレーターとジェネレーター
        - TsDocs・JSDoc リファレンス
        - @types
        - JSX
        - ES6 モジュール
        - ES7 べき乗演算子
        - for-await-of 文
        - new target メタプロパティ
        - 動的 import 式
        - “tsc –watch”

- 非 null アサーション演算子
- デフォルト値を持つ宣言
- オプショナルチェーン
- null 合体演算子
- テンプレートリテラル型
- 関数オーバーロード
- 再帰型
- 再帰条件型
- Node における ECMAScript モジュールのサポート
- アサーション関数
- 可変長タプル型
- ボックス化された型
- TypeScript における共変性と反変性
  - 型パラメーターのオプショナルな変性アノテーション
- テンプレート文字列パターンのインデックスシグネチャ
- satisfies 演算子
- 型のみのインポートとエクスポート
- using 宣言と明示的なリソース管理
  - await using 宣言
- インポート属性
- 正規表現の構文チェック
- import defer

はじめに

『The Concise TypeScript Book』へようこそ！このガイドでは、TypeScript を使って効果的に開発するために不可欠な知識と実践的

なスキルを身につけられます。クリーンで堅牢なコードを書くための主要な概念とテクニックを学びましょう。初心者にも経験豊富な開発者にも、本書はプロジェクトで TypeScript の力を活用するための包括的なガイドであると同時に、手軽なリファレンスとしても役立ちます。

本書は TypeScript 7.0 を扱います。

著者について

Simone Poggiali は、90 年代からプロフェッショナル品質のコードを書くことに情熱を注いできた、経験豊富な Staff Engineer です。国際的なキャリアを通じて、スタートアップから大規模組織まで、幅広いクライアントの数多くのプロジェクトに貢献してきました。HelloFresh、Siemens、O2、Leroy Merlin、Snowplow などの著名な企業が、彼の専門知識と献身から恩恵を受けています。

以下のプラットフォームで Simone Poggiali に連絡できます。

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- メール: gibbok.coding□gmail.com

コントリビューターの全一覧: https://github.com/gibbok/typescript-book/graphs/contributors

TypeScript 入門

TypeScript とは？

TypeScript は、JavaScript を基盤とする強く型付けされたプログラミング言語です。元々は 2012 年に Anders Hejlsberg によって設計され、現在は Microsoft がオープンソースプロジェクトとして開発および保守しています。

TypeScript は JavaScript にコンパイルされ、あらゆる JavaScript ランタイム（たとえばブラウザーやサーバー上の Node.js）で実行できます。

関数型、ジェネリック、命令型、オブジェクト指向プログラミングなど、複数のプログラミングパラダイムをサポートしています。また、実行前に JavaScript へ変換されるコンパイル（トランスパイル）言語です。

TypeScript を選ぶ理由

TypeScript は強く型付けされた言語であり、一般的なプログラミング上の誤りを防ぎ、プログラムの実行前に特定の種類のランタイムエラーを回避するのに役立ちます。

強く型付けされた言語では、開発者がデータ型の定義によってプログラムのさまざまな制約や振る舞いを指定できます。これにより、ソフトウェアの正しさを検証しやすくなり、不具合を防止できます。これは特に大規模なアプリケーションで有用です。

TypeScript の利点の一部を以下に示します。

- 静的型付け（任意で強い型付け）
- 型推論
- ES6 および ES7 の機能を利用可能
- クロスプラットフォームおよびクロスブラウザー互換性

- IntelliSense によるツールサポート

TypeScript と JavaScript

TypeScript は .ts または .tsx ファイルに記述し、JavaScript ファイルは .js または .jsx ファイルに記述します。

拡張子が .tsx または .jsx のファイルには、React の UI 開発で使用される JavaScript 構文拡張の JSX を含めることができます。

構文上、TypeScript は JavaScript（ECMAScript 2015）の型付きスーパーセットです。すべての JavaScript コードは有効な TypeScript コードですが、その逆が常に成り立つとは限りません。

たとえば、次のような .js 拡張子の JavaScript ファイル内の関数を考えてみましょう。

```
const sum = (a, b) => a + b;
```

ファイル拡張子を .ts に変更すれば、この関数を TypeScript に変換して使用できます。ただし、同じ関数に TypeScript の型アノテーションを付けると、コンパイルなしではどの JavaScript ランタイムでも実行できません。次の TypeScript コードは、コンパイルしなければ構文エラーになります。

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript は、開発者が型アノテーションを通じて意図を表現できるようにすることで、潜在的なランタイムエラーをコンパイル時に検出するよう設計されています。さらに、型推論により、明示的な型アノテーションがなくても TypeScript は特定の問題を検出でき

ます。たとえば、次のコードスニペットでは TypeScript の型を一切指定していません。

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

この場合、TypeScript はエラーを検出し、次のように報告します。

```
Property 'y' does not exist on type '{ x: number; }'.
```

TypeScript の型システムは、JavaScript の実行時の振る舞いから大きな影響を受けています。たとえば、JavaScript では文字列の連結または数値の加算を行う加算演算子（+）は、TypeScript でも同じようにモデル化されています。

```
const result = '1' + 1; // 結果の型は string
```

TypeScript の開発チームは、JavaScript の通常とは異なる使い方をエラーとして示すという意図的な判断をしています。たとえば、次の有効な JavaScript コードを考えてみましょう。

```
const result = 1 + true; // JavaScript では結果は 2
```

しかし、TypeScript はエラーを報告します。

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

このエラーが発生するのは、TypeScript が型の互換性を厳密に適用し、この場合は数値と真偽値の間の無効な演算を識別するためです。

TypeScript のコード生成

TypeScript コンパイラには、型エラーのチェックと JavaScript へのコンパイルという 2 つの主要な役割があります。この 2 つの処理は互いに独立しています。型はコンパイル時に完全に消去されるため、JavaScript ランタイムでのコードの実行には影響しません。TypeScript は型エラーが存在していても JavaScript を出力できます。 型エラーを含む TypeScript コードの例を以下に示します。

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // 型 'string' の引数を型 'number' のパラメーターに割り当てることはできません。
```

それでも、実行可能な JavaScript を出力できます。

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

実行時に TypeScript の型をチェックすることはできません。次に例を示します。

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' は型のみを参照しますが、ここでは値として使用されています。
        // ...
    }
};
```

型はコンパイル後に消去されるため、このコードを JavaScript で実行する方法はありません。実行時に型を識別するには、別の仕組みを使用する必要があります。TypeScript には複数の選択肢があり、よく使われるものの 1 つが「タグ付きユニオン」です。次に例を示します。

```
interface Dog {
    kind: 'dog'; // タグ付きユニオン
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // タグ付きユニオン
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

「kind」プロパティは、JavaScript でオブジェクトを区別するために実行時に使用できる値です。

また、実行時の値が、型宣言で宣言された型とは異なる型を持つ可能性もあります。たとえば、開発者が API の型を誤って解釈し、誤った型アノテーションを付けた場合です。

TypeScript は JavaScript のスーパーセットであるため、「class」キーワードは実行時に型および値として使用できます。

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};
```

```
const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

JavaScript では、「class」には「prototype」プロパティがあり、「instanceof」演算子を使用して、コンストラクターの prototype プロパティがオブジェクトのプロトタイプチェーン上のどこかに存在するかどうかを検査できます。

すべての型が消去されるため、TypeScript が実行時のパフォーマンスに影響を与えることはありません。ただし、TypeScript によってビルド時間のオーバーヘッドが多少発生します。

モダン JavaScript を今すぐ使う（ダウンレベリング）

TypeScript は、ECMAScript 3（1999 年）以降にリリースされたあらゆるバージョンの JavaScript へコードをコンパイルできます。つまり、TypeScript は最新の JavaScript 機能を古いバージョンへトランスパイルできます。この処理はダウンレベリングと呼ばれます。これにより、古いランタイム環境との互換性を最大限に維持しながら、モダン JavaScript を使用できます。

古いバージョンの JavaScript へトランスパイルする際、TypeScript が生成するコードには、ネイティブ実装と比べてパフォーマンス上のオーバーヘッドが生じる可能性がある点に注意することが重要です。

TypeScript で使用できるモダン JavaScript 機能の一部を以下に示します。

- AMD 形式の「define」コールバックや CommonJS の「require」文の代わりに ECMAScript モジュールを使用す

る。
- プロトタイプの代わりにクラスを使用する。
- 「var」の代わりに「let」または「const」を使用して変数を宣言する。
- 従来の「for」ループの代わりに「for-of」ループまたは「.forEach」を使用する。
- 関数式の代わりにアロー関数を使用する。
- 分割代入。
- プロパティ名やメソッド名の短縮記法、および算出プロパティ名。
- 関数のデフォルトパラメーター。

これらのモダン JavaScript 機能を活用することで、開発者は TypeScript でより表現力に富んだ簡潔なコードを記述できます。

TypeScript を始める

インストール

Visual Studio Code は TypeScript 言語を十分にサポートしていますが、TypeScript コンパイラは含まれていません。TypeScript コンパイラをインストールするには、npm や yarn などのパッケージマネージャーを使用できます。

```
npm install typescript --save-dev
```

または

```
yarn add typescript --dev
```

チームの全メンバーが同じバージョンの TypeScript を使用できるように、生成されたロックファイルを必ずコミットしてください。

TypeScript コンパイラを実行するには、次のコマンドを使用できます。

```
npx tsc
```

または

```
yarn tsc
```

より予測しやすいビルドプロセスになるため、TypeScript はグローバルではなくプロジェクト単位でインストールすることをお勧めします。ただし、一度限りの用途では次のコマンドを使用できます。

```
npx tsc
```

または、グローバルにインストールします。

```
npm install -g typescript
```

Microsoft Visual Studio を使用している場合、MSBuild プロジェクト向けの NuGet パッケージとして TypeScript を取得できます。NuGet パッケージマネージャーコンソールで、次のコマンドを実行します。

```
Install-Package Microsoft.TypeScript.MSBuild
```

TypeScript のインストール時には、TypeScript コンパイラである「tsc」と、TypeScript のスタンドアロンサーバーである「tsserver」の 2 つの実行ファイルがインストールされます。スタンドアロンサーバーにはコンパイラと言語サービスが含まれており、エディターや IDE がインテリジェントなコード補完を提供するために利用できます。

さらに、Babel（プラグイン経由）や swc など、TypeScript に対応したトランスパイラが複数あります。これらのトランスパイラを使用して、TypeScript コードを別のターゲット言語またはバージョンに変換できます。

TypeScript 7.0 では、コンパイラと言語サービスのネイティブ実装として Go で書き直されました。共有メモリによるマルチスレッド処理やその他の最適化を使用して、フルビルドとエディター機能を高速化し、開発中のフィードバック時間を短縮します。

TypeScript 7.0 の一部のパフォーマンス機能は調整できます。型チェックは `--checkers` を使用して複数のワーカーで並列実行できます。ワーカーを増やすと大規模プロジェクトを高速化できますが、使用するメモリも増えます。再構築された `--watch` モードにより、クロスプラットフォームのファイル監視が改善されています。TypeScript 7.0 には、現時点（2026 年 7 月）ではコンパイラ API が含まれていません。そのため、TypeScript 6.0 の API を引き続き必要とするツールは、`@typescript/typescript6` または npm エイリアスを使用することで TypeScript 7.0 と並行して実行できます。

構成

TypeScript は、tsc の CLI オプション、またはプロジェクトのルートに配置する tsconfig.json という専用の構成ファイルを使用して構成できます。

推奨設定があらかじめ入力された tsconfig.json ファイルを生成するには、次のコマンドを使用できます。

```
tsc --init
```

ローカルで tsc コマンドを実行すると、TypeScript は最も近い tsconfig.json ファイルで指定された構成を使用してコードをコンパイルします。

デフォルト設定で実行する CLI コマンドの例を以下に示します。

```
tsc main.ts // 特定のファイル (main.ts) を JavaScript にコンパイル
tsc src/*.ts // 'src' フォルダー内のすべての .ts ファイルを JavaScript にコンパイル
tsc app.ts util.ts --outfile index.js // 2 つの TypeScript ファイル (app.ts と util.ts) を 1 つの JavaScript ファイル (index.js) にコンパイル
```

TypeScript 構成ファイル

tsconfig.json ファイルは、TypeScript コンパイラ（tsc）の構成に使用します。通常は package.json ファイルとともに、プロジェクトのルートに追加します。

注意事項:

- tsconfig.json は json 形式ですが、コメントを使用できます。
- コマンドラインオプションの代わりに、この構成ファイルを使用することをお勧めします。

次のリンクでは、完全なドキュメントとそのスキーマを確認できます。

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

一般的で便利な構成の一覧を以下に示します。

target

「target」プロパティは、TypeScript コードの出力先またはコンパイル先となる ECMAScript のバージョンを指定するために使用します。モダンブラウザーでは ES6 が適切な選択肢です。注意: ES5 のサポートは TypeScript 6.0 で非推奨となり、TypeScript 7.0 ではサポートされなくなりました。

lib

「lib」プロパティは、コンパイル時に含めるライブラリファイルを指定するために使用します。TypeScript は「target」プロパティで指定された機能の API を自動的に含めますが、特定のニーズに応じて特定のライブラリを除外または選択できます。たとえば、サーバープロジェクトで作業している場合、ブラウザー環境でのみ有用な「DOM」ライブラリを除外できます。

strict

「strict」オプションは、より強力なチェックを有効にして型安全性を向上させます。TypeScript 6.0 以降ではデフォルトで有効です。それ以外の場合は、tsconfig.json で明示的に true に設定する必要があります。「strict」を有効にすると、TypeScript では次のようになります。

- 各ソースファイルで「use strict」を使用するコードを出力します。
- 型チェック処理で「null」と「undefined」を考慮します。
- 型アノテーションがない場合に「any」型の使用を無効にします。

- 「this」式の型が暗黙に「any」となる場合に、エラーを報告します。

module

「module」プロパティは、コンパイルされたプログラムでサポートするモジュールシステムを設定します。実行時には、指定されたモジュールシステムに基づいて依存関係を特定し、実行するためにモジュールローダーが使用されます。

JavaScript で最も一般的に使用されるモジュールローダーは、サーバーサイドアプリケーション向けの Node.js CommonJS と、ブラウザーベースのウェブアプリケーションにおける AMD モジュール向けの RequireJS です。TypeScript は、UMD、System、ESNext、ES2015/ES6、ES2020 など、さまざまなモジュールシステム向けのコードを出力できます。モジュールシステムは、ターゲット環境と、その環境で利用できるモジュール読み込みの仕組みに基づいて選択する必要があります。

注意: 古いモジュールシステム（AMD、UMD、SystemJS）のサポートは TypeScript 6.0 で非推奨となり、TypeScript 7.0 ではサポートされなくなりました。

moduleResolution

「moduleResolution」プロパティは、モジュール解決戦略を指定します。モダンな TypeScript コードには「nodenext」または「bundler」を使用します。「classic」戦略は、古いバージョンの TypeScript（1.6 より前）でのみ使用されます。

esModuleInterop

「esModuleInterop」プロパティを使用すると、「default」プロパティを使用してエクスポートしていない CommonJS モジュールからデフォルトインポートができるようになります。このプロパティは、出力される JavaScript で互換性を確保するための shim を提供します。このオプションを有効にすると、`import MyLibrary from "my-library"` を使用でき、`import * as MyLibrary from "my-library"` を使用する必要がなくなります。

「esModuleInterop」は、破壊的変更を避けるために元々は任意で有効にするものでしたが、長い間推奨されるデフォルトとなっています。これを無効にすると、CommonJS と ESM を併用するときに分かりにくい実行時の問題が発生する可能性があります。注意: TypeScript 6.0 以降、このより安全な相互運用の振る舞いは常に有効です。

TypeScript 6.0 では、一部の古い構成オプションと構文形式が非推奨になったか、古い振る舞いを伴う移行の対象となりました。TypeScript 7.0 では、これらはハードエラーまたは何も行わない振る舞いになります。

ハードエラーまたは何も行わない振る舞いになった非推奨項目は、次のとおりです。

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`

- esModuleInterop または allowSyntheticDefaultImports の無効化
- alwaysStrict の無効化
- 名前空間宣言内の module キーワード
- インポートでの asserts
- /// <reference no-default-lib /> （skipDefaultLibCheck の下）
- ローカルの tsconfig.json を伴う CLI ファイルパス（--ignoreConfig を使用する場合を除く）

jsx

「jsx」プロパティは、ReactJS で使用される .tsx ファイルにのみ適用され、JSX 構造を JavaScript にコンパイルする方法を制御します。一般的なオプションは「preserve」です。これは JSX を変更せずに保持した .jsx ファイルへコンパイルするため、そのファイルを Babel などのさまざまなツールに渡して、さらに変換できます。

skipLibCheck

「skipLibCheck」プロパティを使用すると、インポートされたサードパーティパッケージ全体を TypeScript が型チェックしなくなります。このプロパティによって、プロジェクトのコンパイル時間が短縮されます。TypeScript は引き続き、これらのパッケージが提供する型定義に対してコードをチェックします。

files

「files」プロパティは、プログラムに常に含める必要があるファイルの一覧をコンパイラに示します。

include

「include」プロパティは、含めたいファイルの一覧をコンパイラに示します。このプロパティでは、任意のサブディレクトリを表す「*」、*任意のファイル名を表す*「」、オプショナルな文字を表す「?」など、glob に似たパターンを使用できます。

exclude

「exclude」プロパティは、コンパイルに含めるべきでないファイルの一覧をコンパイラに示します。これには「node_modules」やテストファイルなどを含めることができます。 注意: tsconfig.json ではコメントを使用できます。

importHelpers

TypeScript は、特定の高度な JavaScript 機能やダウンレベル化された JavaScript 機能のコードを生成するときに、ヘルパーコードを使用します。デフォルトでは、これらのヘルパーは使用するファイルごとに重複して含まれます。`importHelpers` オプションを使用すると、代わりに `tslib` モジュールからこれらのヘルパーをインポートし、JavaScript の出力をより効率的にできます。

TypeScript への移行に関するアドバイス

大規模なプロジェクトでは、TypeScript コードと JavaScript コードが当初は共存する段階的な移行を採用することをお勧めします。TypeScript へ一度に移行できるのは小規模なプロジェクトだけです。

この移行の最初のステップは、ビルドチェーンの処理に TypeScript を導入することです。これは「allowJs」コンパイラオプションを使用して実現できます。このオプションにより、.ts および .tsx ファイ

ルを既存の JavaScript ファイルと共存させることができます。TypeScript は JavaScript ファイルから型を推論できない変数に対して「any」型へフォールバックするため、移行の開始時にはコンパイラオプションの「noImplicitAny」を無効にすることをお勧めします。

2 番目のステップは、各モジュールを変換しながらテストを実行できるように、JavaScript のテストが TypeScript ファイルとともに動作することを確認することです。Jest を使用している場合は、Jest で TypeScript プロジェクトをテストできる ts-jest の使用を検討してください。

3 番目のステップは、サードパーティライブラリの型宣言をプロジェクトに含めることです。これらの宣言は、ライブラリに同梱されているか、DefinitelyTyped で見つけることができます。https://www.typescriptlang.org/dt/search を使用して検索し、次のコマンドでインストールできます。

```
npm install --save-dev @types/package-name
```

または

```
yarn add --dev @types/package-name
```

4 番目のステップは、依存関係グラフの末端から始め、ボトムアップのアプローチでモジュールごとに移行することです。ほかのモジュールに依存しないモジュールから変換を始めるという考え方です。依存関係グラフを可視化するには、「madge」ツールを使用できます。

このような初期変換に適したモジュールとしては、ユーティリティ関数や、外部 API または仕様に関連するコードが挙げられます。

Swagger コントラクト、GraphQL、または JSON スキーマから TypeScript の型定義を自動生成し、プロジェクトに組み込むことが可能です。

仕様や公式スキーマが存在しない場合は、サーバーから返される JSON などの生データから型を生成できます。ただし、エッジケースの見落としを避けるため、データではなく仕様から型を生成することを推奨します。

移行中はコードのリファクタリングを控え、モジュールへの型の追加のみに集中してください。

5 番目のステップは「noImplicitAny」を有効にすることです。これにより、すべての型が既知で定義済みであることが強制され、プロジェクトでより良い TypeScript 開発体験を得られます。

移行中は、JavaScript ファイルで TypeScript の型チェックを有効にする `@ts-check` ディレクティブを使用できます。このディレクティブは緩やかな型チェックを提供し、最初の段階で JavaScript ファイル内の問題を特定するために使用できます。ファイルに `@ts-check` が含まれている場合、TypeScript は JSDoc 形式のコメントを使用して定義を推論しようとします。ただし、JSDoc アノテーションの使用は、移行のごく初期段階に限ることを検討してください。

tsconfig.json の `noEmitOnError` は、デフォルト値の false のままにすることを検討してください。これにより、エラーが報告された場合でも JavaScript ソースコードを出力できます。

型システムを探る

TypeScript Language Service

tsserver とも呼ばれる TypeScript Language Service は、エラー報告、診断、保存時のコンパイル、名前変更、定義への移動、補完リスト、シグネチャヘルプなど、さまざまな機能を提供します。主に統合開発環境（IDE）で IntelliSense サポートを提供するために使用されます。Visual Studio Code とシームレスに統合され、Conquer of Completion（Coc）などのツールでも利用されています。

開発者は専用 API を活用して独自のカスタム Language Service プラグインを作成し、TypeScript の編集エクスペリエンスを向上させることができます。これは、特殊な lint 機能を実装したり、カスタムテンプレート言語の自動補完を有効にしたりする場合に特に役立ちます。

実際に使われているカスタムプラグインの例として、styled components の CSS プロパティに対する構文エラー報告と IntelliSense サポートを提供する「typescript-styled-plugin」があります。

詳細情報とクイックスタートガイドについては、GitHub 上の公式 TypeScript Wiki を参照してください：https://github.com/microsoft/TypeScript/wiki/

構造的型付け

TypeScript は構造的型システムに基づいています。これは、C# や C の公称型システムのように型の名前や宣言場所ではなく、型の実際の構造や定義によって型の互換性と同等性が決まることを意味します。

TypeScript の構造的型システムは、実行時における JavaScript の動的なダックタイピングシステムの仕組みに基づいて設計されています。

次の例は有効な TypeScript コードです。ご覧のとおり、「X」と「Y」は宣言名が異なりますが、同じメンバー「a」を持っています。型は構造によって決まり、この場合は構造が同じであるため、互換性があり有効です。

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // 有効
```

TypeScript の基本的な比較規則

TypeScript の比較処理は再帰的であり、どの階層にネストされた型に対しても実行されます。

「Y」が少なくとも「X」と同じメンバーを持つ場合、型「X」は「Y」と互換性があります。

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // 有効。少なくとも X と同じメンバーを持つため
const r: X = y;
```

関数のパラメーターは名前ではなく型によって比較されます。

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // 有効
x = y; // 有効
```

関数の戻り値の型は同じでなければなりません。

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // 無効
x = y; // 無効
```

ソース関数の戻り値の型は、ターゲット関数の戻り値の型のサブタイプでなければなりません。

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // 有効
y = x; // 無効。メンバー b がないため
```

関数のパラメーターを破棄することは許可されています。これはJavaScriptで一般的な慣行であり、たとえば「Array.prototype.map()」で使用されています。

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

したがって、次の型宣言は完全に有効です。

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
```

```
let y: Y = (a: number) => undefined; // パラメーター b がない
y = x; // 有効
```

ソース型に追加された任意のオプショナルパラメーターは有効です。

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // 有効
x = y; //有効
```

ソース型に対応するパラメーターがないターゲット型のオプショナルパラメーターはすべて有効であり、エラーにはなりません。

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // 有効
x = y; // 有効
```

rest パラメーターは、オプショナルパラメーターが無限に続くものとして扱われます。

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //有効
```

オーバーロードを持つ関数は、オーバーロードシグネチャが実装シグネチャと互換性がある場合に有効です。

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
```

```
    console.log(a, b);
}
x('a'); // 有効
x('a', 1); // 有効

function y(a: string): void; // 無効。実装シグネチャと互換性がないため
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

ソースとターゲットのパラメーターがスーパータイプまたはサブタイプに代入可能な場合、関数パラメーターの比較は成功します（双変性）。

```
// スーパータイプ
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// サブタイプ
class Y extends X {}
// サブタイプ
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// 双変性ではスーパータイプも受け入れる
console.log(getA(new X('x'))); // 有効
```

```
console.log(getA(new Y('Y'))); // 有効
console.log(getA(new Z('z'))); // 有効
```

列挙型は数値と比較でき、その逆も有効ですが、異なる列挙型の列挙値どうしを比較することは無効です。

```
enum X {
    A,
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // 有効
const ya: Y = 0; // 有効
X.A === Y.A; // 無効
```

クラスのインスタンスでは、private および protected メンバーに対して互換性チェックが行われます。

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}
```

```
let x: X = new Y('y'); // 無効
```

比較チェックでは、継承階層の違いは考慮されません。たとえば次のとおりです。

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // 有効
x === z; // z が異なる継承階層に属していても有効
```

ジェネリクスは、ジェネリック型パラメーターを適用した後に得られる型の構造に基づいて比較され、最終結果だけが非ジェネリック型として比較されます。

```ts
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // 無効。最終的な構造で型引数が使用されているため

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // 有効。最終的な構造で型引数が使用されていないため
```

ジェネリクスの型引数が指定されていない場合、指定されていないすべての引数は「any」型として扱われます。

```ts
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // 有効
```

次の点を覚えておいてください。

```ts
let a: number = 1;
let b: number = 2;
a = b; // 有効。すべての型はそれ自身に割り当て可能

let c: any;
c = 1; // 有効。すべての型は any に割り当て可能

let d: unknown;
d = 1; // 有効。すべての型は unknown に割り当て可能

let e: unknown;
let e1: unknown = e; // 有効。unknown はそれ自身と any にのみ割り当て可能
let e2: any = e; // 有効
```

```
let e3: number = e; // 無効

let f: never;
f = 1; // 無効。never には何も割り当てられない

let g: void;
let g1: any;
g = 1; // 無効。void は any 以外の型との相互の割り当てができない
g = g1; // 有効
```

「strictNullChecks」が有効な場合、「null」と「undefined」は「void」と同様に扱われます。それ以外の場合は「never」と同様に扱われることに注意してください。

集合としての型

TypeScript では、型は取り得る値の集合です。この集合は型のドメインとも呼ばれます。型の各値は、集合内の要素とみなすことができます。型は、集合内のすべての要素がその集合のメンバーとみなされるために満たすべき制約を定めます。 TypeScript の主な役割は、ある集合が別の集合の部分集合であるかをチェックし、検証することです。

TypeScript は、さまざまな種類の集合をサポートしています。

集合の用語	TypeScript	注記
空集合	never	「never」はそれ自体以外のあらゆるものを含む
単一要素の集合	undefined / null / リテラル型	
有限集合	boolean / union	

無限集合	string / number / object	
全体集合	any / unknown	すべての要素は「any」のメンバーで、すべての集合はその部分集合である / 「unknown」は「any」の型安全な対となる型

いくつか例を示します。

TypeScript	集合の用語	例
never	∅（空集合）	const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’
リテラル型	単一要素の集合	type X = ‘X’;
		type Y = 7;
T に代入可能な値	値 ∈ T（メンバー）	type XY = ‘X’ | ‘Y’;
		const x: XY = ‘X’;
T2 に代入可能な T1	T1 ⊆ T2（部分集合）	type XY = ‘X’ | ‘Y’;
		const x: XY = ‘X’;
		const j: XY = ‘J’; // Type ‘“J”’ is not assignable to type ‘XY’.
T1 extends T2	T1 ⊆ T2（部分集合）	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2（和集合）	type XY = ‘X’ | ‘Y’;
		type JK = 1 | 2;
T1 & T2	T1 ∩ T2（共通部分）	type X = { a: string }
		type Y = { b: string }
		type XY = X & Y
		const x: XY = { a: ‘a’, b: ‘b’ }

unknown　　全体集合　　const x: unknown = 1

ユニオン（T1 | T2）は、より広い集合（両方）を作成します。

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // 有効
```

交差型（T1 & T2）は、より狭い集合（共有されるものだけ）を作成します。

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // 無効
const j: XY = { a: 'a', b: 'b' }; // 有効
```

この文脈では、extends キーワードは「部分集合」と考えることができます。これは型に制約を設定します。extends をジェネリックとともに使用すると、ジェネリック型パラメーターがより具体的な型に制約されます。

ここでの extends は、OOP の意味でのクラス継承とはまったく関係がないことに注意してください。

TypeScript は構造的型を扱い、厳密な公称型階層を持ちません。実際、次の例のように、TypeScript はオブジェクトの構造、つまり形状を考慮するため、2 つの型はどちらも他方のサブタイプではなくても重複することがあります。

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // 有効
```

型を割り当てる：型宣言と型アサーション

TypeScript では、さまざまな方法で型を割り当てることができます。

型宣言

次の例では、x: X（“: Type”）を使用して変数 x の型を宣言します。

```
type X = {
    a: string;
};

// 型宣言
const x: X = {
    a: 'a',
};
```

変数が指定された形式でない場合、TypeScript はエラーを報告します。たとえば次のとおりです。

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
    b: 'b', // エラー: オブジェクトリテラルで指定できるのは既知のプロパティのみ
};
```

型アサーション

as キーワードを使用してアサーションを追加できます。これは、開発者が型についてより多くの情報を持っていることをコンパイラに

伝え、発生し得るエラーを抑制します。

例：

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

上の例では、as キーワードを使用して、オブジェクト x が型 X であるとアサーションしています。これにより、型定義に存在しない追加のプロパティ b があっても、オブジェクトが指定された型に準拠していることを TypeScript コンパイラに伝えます。

型アサーションは、より具体的な型を指定する必要がある状況、特に DOM を扱う場合に役立ちます。たとえば次のとおりです。

```
const myInput = document.getElementById('my_input') as HTMLInputElement;
```

ここでは、型アサーション as HTMLInputElement を使用して、getElementById の結果を HTMLInputElement として扱うよう TypeScript に伝えています。 次のテンプレートリテラルの例に示すように、型アサーションをキーの再マッピングに使用することもできます。

```
type J<Type> = {
    [Property in keyof Type as `prefix_${string &
        Property}`]: () => Type[Property];
};
type X = {
    a: string;
```

```
    b: number;
};
type Y = J<X>;
```

この例では、型 J<Type> はテンプレートリテラルを持つマップ型を使用して Type のキーを再マッピングします。各キーに「prefix_」を追加した新しいプロパティを作成し、対応する値は元のプロパティ値を返す関数になります。

型アサーションを使用すると、TypeScript は過剰プロパティチェックを実行しない点に注意してください。そのため、オブジェクトの構造が事前に分かっている場合は、通常、型宣言を使用することが望ましいです。

アンビエント宣言

アンビエント宣言は JavaScript コードの型を記述するファイルで、ファイル名の形式は .d.ts. です。通常は、既存の JavaScript ライブラリに型アノテーションを付けたり、プロジェクト内の既存の JS ファイルに型を追加したりするためにインポートして使用します。

一般的なライブラリの型の多くは、次の場所で確認できます。
https://github.com/DefinitelyTyped/DefinitelyTyped/

また、次のコマンドでインストールできます。

```
npm install --save-dev @types/library-name
```

独自に定義したアンビエント宣言は、「トリプルスラッシュ」リファレンスを使用してインポートできます。

```
/// <reference path="./library-types.d.ts" />
```

// @ts-check を使用すれば、JavaScript ファイル内でもアンビエント宣言を使用できます。

declare キーワードを使用すると、既存の JavaScript コードをインポートせずに型定義を記述でき、別のファイルまたはグローバルな型のプレースホルダーとして機能します。

プロパティチェックと過剰プロパティチェック

TypeScript は構造的型システムに基づいていますが、過剰プロパティチェックは、オブジェクトが型で指定されたプロパティを正確に持つかどうかをチェックできる TypeScript の機能です。

過剰プロパティチェックは、たとえばオブジェクトリテラルを変数に代入するときや、関数の引数として渡すときに実行されます。

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
const x: X = y; // 構造的型付けにより有効
const w: X = { a: 'a', b: 'b' }; // 過剰プロパティチェックにより無効
```

弱い型

すべてオプショナルなプロパティの集合だけで構成される型は、弱い型とみなされます。

```
type X = {
    a?: string;
    b?: string;
};
```

弱い型と共通するプロパティがない値を代入すると、TypeScript はエラーとみなします。たとえば、次のコードはエラーを発生させます。

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // 無効
```

推奨はされませんが、必要であれば、型アサーションを使用してこのチェックを回避できます。

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // 有効
```

または、unknown を値の型とするインデックスシグネチャを弱い型に追加します。

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // 有効
```

厳密なオブジェクトリテラルチェック（Freshness）

厳密なオブジェクトリテラルチェックは「freshness」と呼ばれることもあり、通常の構造的型チェックでは見落とされてしまう過剰なプロパティやスペルミスのあるプロパティを検出するのに役立つ TypeScript の機能です。

オブジェクトリテラルを作成すると、TypeScript コンパイラはそれを「fresh」とみなします。オブジェクトリテラルを変数に代入するかパラメーターとして渡した場合、そのオブジェクトリテラルにターゲット型に存在しないプロパティが指定されていると、TypeScript はエラーを発生させます。

ただし、オブジェクトリテラルの型が拡大された場合や、型アサーションが使用された場合は、「freshness」が失われます。

これを示す例をいくつか紹介します。

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Freshness チェック: 無効な代入
var y: Y;
y = { a: 'a', bx: 'bx' }; // Freshness チェック: 無効な代入

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // 型の拡大: エラーなし。構造的に型の互換性がある

fn({ a: 'a', bx: 'b' }); // Freshness チェック: 無効な引数

let c: X = { a: 'a' };
```

```
let d: Y = { a: 'a', b: '' };
c = d; // 型の拡大: Freshness チェックなし
```

型推論

TypeScript は、次の場合にアノテーションが指定されていなくても型を推論できます。

- 変数の初期化。
- メンバーの初期化。
- パラメーターのデフォルト値の設定。
- 関数の戻り値の型。

例：

```
let x = 'x'; // 推論される型は string
```

TypeScript コンパイラは値または式を分析し、利用可能な情報に基づいてその型を決定します。

より高度な推論

型推論で複数の式が使用されている場合、TypeScript は「最適な共通型」を探します。たとえば次のとおりです。

```
let x = [1, 'x', 1, null]; // 推論される型: (string | number | null)[]
```

コンパイラが最適な共通型を見つけられない場合は、ユニオン型を返します。例：

```
let x = [new RegExp('x'), new Date()]; // 推論される型: (RegExp | Date)[]
```

TypeScript は、変数の位置に基づく「コンテキスト型付け」を利用して型を推論します。次の例では、コンパイラは `e` が `MouseEvent` 型で

あることを認識します。これは、さまざまな一般的な JavaScript 構文と DOM のアンビエント宣言を含む lib.d.ts ファイルに `click` イベント型が定義されているためです。

```
window.addEventListener('click', function (e) {}); // e の推論される型は MouseEvent
```

型の拡大

型の拡大は、型アノテーションなしで初期化された変数に TypeScript が型を割り当てるプロセスです。狭い型から広い型への変換は許可しますが、その逆は許可しません。 次の例では、

```
let x = 'x'; // TypeScript は広い型である string と推論
let y: 'y' | 'x' = 'y'; // y の型はリテラル型のユニオン
y = x; // 無効。型 'string' を型 '"x" | "y"' に割り当てることはできません。
```

TypeScript は `string` を `x` に割り当てます。これは、初期化時に指定された単一の値（x）に基づく型の拡大の一例です。

TypeScript には、たとえば「const」を使用するなど、型の拡大のプロセスを制御する方法があります。

const

変数を宣言するときに `const` キーワードを使用すると、TypeScript ではより狭い型が推論されます。

例：

```
const x = 'x'; // TypeScript は x の型をより狭い型である 'x' と推論
let y: 'y' | 'x' = 'y';
y = x; // 有効: x の型は 'x' と推論される
```

const を使用して変数 x を宣言すると、その型は特定のリテラル値 'x' に絞り込まれます。x の型が絞り込まれているため、エラーなしで変数 y に代入できます。 型を推論できる理由は、const 変数は再代入できず、その型を特定のリテラル型、この場合はリテラル型 'x' に絞り込めるためです。

型パラメーターの const 修飾子

TypeScript のバージョン 5.0 以降では、ジェネリック型パラメーターに const 修飾子を指定できます。これにより、可能な限り正確な型を推論できます。まず、const を使用しない例を見てみましょう。

```
function identity<T>(value: T) {
    // ここでは const を使用しない
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // 推論される型: { a: string; b: string; }
```

ご覧のとおり、プロパティ a と b は string 型として推論されています。

次に、const を使用したバージョンとの違いを見てみましょう。

```
function identity<const T>(value: T) {
    // 型パラメーターに const 修飾子を使用
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // 推論される型: { a: "a"; b: "b"; }
```

プロパティ a と b が、単なる string 型ではなく文字列リテラルとして推論されていることが分かります。

const アサーション

この機能を使用すると、初期値に基づくより正確なリテラル型で変数を宣言し、その値を不変のリテラルとして扱うべきであることをコンパイラに示せます。いくつか例を示します。

単一のプロパティの場合：

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

オブジェクト全体の場合：

```
const v = {
    x: 1,
    y: 2,
} as const;
```

これは、タプルの型を定義するときに特に役立ちます。

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // readonly [1, 2, 3] 型のタプル
```

明示的な型アノテーション

具体的な型を渡すことができます。次の例では、プロパティ x は number 型です。

```
const v = {
    x: 1, // 推論される型: number (型の拡大)
};
v.x = 3; // 有効
```

リテラル型のユニオンを使用すると、型アノテーションをより具体的にできます。

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x はリテラル型のユニオンになる: 1 | 2 | 3
};
v.x = 3; // 有効
v.x = 100; // 無効
```

型の絞り込み

型の絞り込みとは、TypeScript で一般的な型をより具体的な型に絞り込むプロセスです。これは、TypeScript がコードを分析し、特定の条件や操作によって型情報を精緻化できると判断した場合に行われます。

型の絞り込みは、次のようなさまざまな方法で行えます。

条件

`if` や `switch` などの条件文を使用することで、TypeScript は条件の結果に基づいて型を絞り込めます。例：

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // 型は number。条件によって絞り込まれている
}
```

throw または return

エラーを throw したり、分岐から早期 return したりすることで、TypeScript による型の絞り込みを支援できます。例：

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

TypeScript で型を絞り込むその他の方法には、次のものがあります。

- instanceof 演算子：オブジェクトが特定のクラスのインスタンスであるかをチェックするために使用します。
- in 演算子：オブジェクトにプロパティが存在するかをチェックするために使用します。
- typeof 演算子：実行時に値の型をチェックするために使用します。
- Array.isArray() などの組み込み関数：値が配列かどうかをチェックするために使用します。

判別可能なユニオン

「判別可能なユニオン」は、ユニオン内の異なる型を区別するためにオブジェクトへ明示的な「タグ」を追加する TypeScript のパターンです。このパターンは「タグ付きユニオン」とも呼ばれます。次の例では、「タグ」はプロパティ「type」で表されます。

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // 型は A
```

```
        case 'type_b':
            return input.value + 'extra'; // 型は B
    }
};
```

ユーザー定義型ガード

TypeScript が型を判定できない場合、「ユーザー定義型ガード」と呼ばれるヘルパー関数を記述できます。次の例では、型述語を利用し、特定のフィルタリングを適用した後で型を絞り込みます。

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // 型は (string | null)[]。TypeScript は型を適切に推論できなかった

const isValid = (item: string | null): item is string => item !== null; // カスタム型ガード

const r2 = data.filter(isValid); // 型は適切な string[] になる。型述語による型ガードで型を絞り込めた
```

switch-true による絞り込み

TypeScript 5.3 では switch-true による絞り込みが追加され、煩雑な if/else チェーンを、真偽値条件を使用した switch (true) に置き換えられるようになりました。可読性が向上し、型の絞り込みも維持されます。パターンマッチングに似ていますが、よりシンプルです。

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
```

```
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

プリミティブ型

TypeScript は 7 種類のプリミティブ型をサポートしています。プリミティブデータ型とは、オブジェクトではなく、関連付けられたメソッドを持たない型を指します。TypeScript では、すべてのプリミティブ型は不変です。つまり、一度代入された値は変更できません。

string

string プリミティブ型はテキストデータを格納し、値は常にダブルクォートまたはシングルクォートで囲まれます。

```
const x: string = 'x';
const y: string = 'y';
```

バッククォート（`）文字で囲むと、文字列を複数行にわたって記述できます。

```
let sentence: string = `xxx,
  yyy`;
```

boolean

TypeScript の boolean データ型は、true または false のいずれかの二値を格納します。

```
const isReady: boolean = true;
```

number

TypeScript の number データ型は、64 ビット浮動小数点値で表されます。number 型は整数と小数を表現できます。 TypeScript は、たとえば次のように、16 進数、2 進数、8 進数もサポートしています。

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // 16 進数は 0x で始まる
const binary: number = 0b1010; // 2 進数は 0b で始まる
const octal: number = 0o633; // 8 進数は 0o で始まる
```

bigint

bigint は、number がサポートする最大安全整数である 2^53 - 1 を超える整数値も表せます。

bigint は、組み込み関数 BigInt() を呼び出すか、任意の整数リテラルの末尾に n を追加することで作成できます。

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

注記：

- bigint 値は number と混在できず、組み込みの Math とともに使用することもできません。同じ型に強制変換する必要があります。

- bigint 値を使用できるのは、target の設定が ES2020 以上の場合のみです。

Symbol

Symbol は、命名の競合を防ぐためにオブジェクトのプロパティキーとして使用できる一意の識別子です。

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null と undefined

null 型と undefined 型は、どちらも値がないこと、または値が存在しないことを表します。

undefined 型は、値が代入も初期化もされていないこと、または意図しない値の不在を示します。

null 型は、そのフィールドに値がないことが分かっているため値を利用できないことを意味し、意図的な値の不在を示します。

Array

array は、同じ型または異なる型の複数の値を格納できるデータ型です。次の構文を使用して定義できます。

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // ユニオン
```

TypeScript は、次の構文を使用した読み取り専用配列をサポートしています。

```
const x: readonly string[] = ['a', 'b']; // readonly 修飾子
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // 無効
```

TypeScript はタプルと読み取り専用タプルをサポートしています。

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

any データ型は文字どおり「あらゆる」値を表し、TypeScript が型を推論できない場合や型が指定されていない場合のデフォルトです。

any を使用すると TypeScript コンパイラは型チェックを省略するため、any の使用時には型安全性がありません。一般に、エラーが発生したときにコンパイラのチェックを抑制するために any を使用しないでください。代わりにエラーの修正に集中してください。any を使用するとコントラクトが破られ、TypeScript の自動補完の恩恵が失われる可能性があります。

any 型は、JavaScript から TypeScript への段階的な移行中に、コンパイラのチェックを抑制するために役立つことがあります。

新規プロジェクトでは、TypeScript 設定 noImplicitAny を使用してください。これにより、any が使用または推論された箇所で TypeScript がエラーを出せるようになります。

any 型は通常、型に関する実際の問題を覆い隠す可能性があるエラーの原因です。できる限り使用を避けてください。

型アノテーション

var、let、const を使用して宣言した変数には、必要に応じて型を追加できます。

```
const x: number = 1;
```

TypeScript は、特に単純な型については型推論を適切に行うため、ほとんどの場合、これらの宣言は必要ありません。

関数では、パラメーターに型アノテーションを追加できます。

```
function sum(a: number, b: number) {
    return a + b;
}
```

次は、無名関数（ラムダ関数とも呼ばれます）を使用した例です。

```
const sum = (a: number, b: number) => a + b;
```

パラメーターにデフォルト値がある場合、これらのアノテーションは省略できます。

```
const sum = (a = 10, b: number) => a + b;
```

関数には戻り値の型アノテーションを追加できます。

```
const sum = (a = 10, b: number): number => a + b;
```

これは、より複雑な関数で特に有用です。実装の前に戻り値の型を記述することで、関数について考えを整理しやすくなります。

一般には、型シグネチャにはアノテーションを付ける一方、関数本体内のローカル変数には付けず、オブジェクトリテラルには常に型を追加することを検討してください。

オプショナルプロパティ

オブジェクトでは、プロパティ名の末尾に疑問符 ? を追加することで、オプショナルプロパティを指定できます。

```
type X = {
    a: number;
    b?: number; // オプショナル
};
```

プロパティがオプショナルの場合、デフォルト値を指定することもできます。

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

読み取り専用プロパティ

修飾子 readonly を使用すると、プロパティへの書き込みを防止できます。これにより、そのプロパティが再度書き換えられないことは保証されますが、完全な不変性が保証されるわけではありません。

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

インデックスシグネチャ

TypeScript では、インデックスシグネチャとして string、number、symbol を使用できます。

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // k[1] と同じ結果
```

JavaScript は number のインデックスを自動的に string のインデックスへ変換するため、k[1] と k["1"] は同じ値を返すことに注意してください。

型の拡張

interface を拡張して、別の型からメンバーをコピーできます。

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

複数の型から拡張することもできます。

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

extends キーワードはインターフェースとクラスでのみ機能します。型では交差型を使用します。

```
type A = {
    a: number;
};
type B = {
```

```
    b: number;
};
type C = A & B;
```

インターフェースを使用して型を拡張することはできますが、その逆はできません。

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

リテラル型

リテラル型は、集合型の中にある要素が 1 つだけの集合です。JavaScript のプリミティブである、非常に限定された値を定義します。

TypeScript のリテラル型には、数値、文字列、真偽値があります。

リテラルの例を次に示します。

```
const a = 'a'; // 文字列リテラル型
const b = 1; // 数値リテラル型
const c = true; // 真偽値リテラル型
```

文字列、数値、真偽値のリテラル型は、ユニオン、型ガード、型エイリアスで使用されます。 次の例では、ユニオン型エイリアスを確認できます。O は指定された値のみで構成され、ほかの文字列は有効ではありません。

```
type O = 'a' | 'b' | 'c';
```

リテラル推論

リテラル推論は、変数またはパラメーターの型を、その値に基づいて推論できる TypeScript の機能です。

次の例では、値を後から変更できないため、TypeScript が x をリテラル型と見なすことが分かります。一方、y は後からいつでも変更できるため、文字列として推論されます。

```
const x = 'x'; // この値は変更できないため、リテラル型 'x'
let y = 'y'; // この値は変更できるため、型は string
```

次の例では、TypeScript が値は後からいつでも変更できると見なすため、o.x が string として（a のリテラルとしてではなく）推論されたことが分かります。

```
type X = 'a' | 'b';

let o = {
    x: 'a', // これはより広い string 型
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // 型 'string' の引数を型 'X' のパラメーターに割り当てることはできません
```

ご覧のとおり、X はより狭い型であるため、o.x を fn に渡すとコードはエラーを発生させます。

この問題は、const または X 型による型アサーションを使用することで解決できます。

```
let o = {
    x: 'a' as const,
};
```

または、次のようにします。

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

strictNullChecks は、厳密な null チェックを強制する TypeScript のコンパイラオプションです。このオプションを有効にすると、変数とパラメーターには、ユニオン型 null | undefined を使用して明示的にその型として宣言されている場合にのみ、null または undefined を代入できます。変数またはパラメーターが null 許容として明示的に宣言されていない場合、潜在的なランタイムエラーを防ぐために TypeScript がエラーを生成します。

列挙型

TypeScript の enum は、名前付き定数値の集合です。

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

列挙型はさまざまな方法で定義できます。

数値列挙型

TypeScript の数値列挙型とは、各定数に数値が割り当てられ、デフォルトでは 0 から始まる列挙型です。

```
enum Size {
    Small, // 値は 0 から始まる
    Medium,
    Large,
}
```

値を明示的に割り当てることで、独自の値を指定できます。

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

文字列列挙型

TypeScript の文字列列挙型とは、各定数に文字列値が割り当てられた列挙型です。

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

注: TypeScript では、文字列メンバーと数値メンバーが共存する異種列挙型を使用できます。

定数列挙型

TypeScript の定数列挙型は、すべての値がコンパイル時に判明しており、列挙型が使用されるすべての箇所にインライン展開される特殊な列挙型で、より効率的なコードになります。

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

次のようにコンパイルされます。

```
console.log('EN' /* Language.English */);
```

注: 定数列挙型では値がハードコードされ、列挙型が消去されます。この方法は自己完結型ライブラリでは効率が高くなる可能性がありますが、一般には望ましくありません。また、定数列挙型に計算メンバーを含めることはできません。

逆引きマッピング

TypeScript の列挙型における逆引きマッピングとは、値から列挙型メンバー名を取得できる機能を指します。デフォルトでは、列挙型メンバーには名前から値への順方向マッピングがありますが、各メンバーの値を明示的に設定することで逆引きマッピングを作成できます。逆引きマッピングは、値から列挙型メンバーを検索する必要がある場合や、列挙型のすべてのメンバーを反復処理する必要がある場合に便利です。数値の列挙型メンバーのみ逆引きマッピングが生成され、文字列の列挙型メンバーには逆引きマッピングがまったく生成されないことに注意してください。

次の列挙型は、

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

次のようにコンパイルされます。

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

したがって、値からキーへのマッピングは数値の列挙型メンバーでは機能しますが、文字列の列挙型メンバーでは機能しません。

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]); // インデックス式が型 'number' ではないため、
要素は暗黙的に 'any' 型になります。
```

アンビエント列挙型

TypeScript のアンビエント列挙型は、関連する実装を持たずに宣言ファイル（*.d.ts）で定義される列挙型の一種です。これにより、各ファイルに実装の詳細をインポートすることなく、異なるファイル間で型安全に使用できる名前付き定数の集合を定義できます。

計算メンバーと定数メンバー

TypeScript の計算メンバーは実行時に値が計算される列挙型のメンバーであり、定数メンバーはコンパイル時に値が設定され、実行時には変更できないメンバーです。計算メンバーは通常の列挙型で使用でき、定数メンバーは通常の列挙型と const 列挙型の両方で使用できます。

```
// 定数メンバー
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 コンパイル時に生成

// 計算メンバー
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // 実行時に生成される乱数
```

列挙型は、そのメンバー型から構成されるユニオンで表されます。各メンバーの値は定数式または非定数式によって決定でき、定数値

を持つメンバーにはリテラル型が割り当てられます。例として、型 E とそのサブタイプ E.A、E.B、E.C の宣言を考えてみましょう。この場合、E はユニオン E.A | E.B | E.C を表します。

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // 数値リテラル
    B = 'bar', // 文字列リテラル
    C = identity(42), // 不透明な計算値
}

console.log(E.C); //42
```

型の絞り込み

TypeScript の型の絞り込みは、条件ブロック内で変数の型をより具体的にするプロセスです。これは、変数が複数の型を持つ可能性があるユニオン型を扱う際に役立ちます。

TypeScript は、型を絞り込むいくつかの方法を認識します。

typeof 型ガード

typeof 型ガードは、組み込みの JavaScript 型に基づいて変数の型を確認する、TypeScript の型ガードの 1 つです。

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x は number
    }
    return -1;
};
```

真偽性による絞り込み

TypeScript の真偽性による絞り込みは、変数が truthy か falsy かを確認し、それに応じて型を絞り込む仕組みです。

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

等価性による絞り込み

TypeScript の等価性による絞り込みは、変数が特定の値と等しいかどうかを確認し、それに応じて型を絞り込む仕組みです。

型を絞り込むために、switch 文や ===、!==、==、!= などの等価演算子と組み合わせて使用します。

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

in 演算子による絞り込み

TypeScript の in 演算子による絞り込みは、変数の型にプロパティが存在するかどうかに基づいて、その変数の型を絞り込む方法で

す。

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

instanceof による絞り込み

TypeScript の instanceof 演算子による絞り込みは、オブジェクトが特定のクラスまたはインターフェースのインスタンスであるかどうかを確認し、そのコンストラクター関数に基づいて変数の型を絞り込む方法です。

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
```

```
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

代入

TypeScript で代入を使用した型の絞り込みは、変数に代入された値に基づいて、その変数の型を絞り込む方法です。変数に値が代入されると、TypeScript は代入された値に基づいて型を推論し、推論された型に一致するように変数の型を絞り込みます。

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

制御フロー解析

TypeScript の制御フロー解析は、コードの流れを静的に解析して変数の型を推論する方法です。これにより、コンパイラは解析結果に

基づいて必要に応じてそれらの変数の型を絞り込めます。

TypeScript 4.4 より前では、コードフロー解析は if 文内のコードにのみ適用されていましたが、TypeScript 4.4 以降では、const 変数を介して間接的に参照される条件式や判別プロパティへのアクセスにも適用できます。

例:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
    obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

型の絞り込みが行われない例をいくつか示します。

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
        x.length; // エラー。isString が const ではないため絞り込まれない
    }
};
```

```
const f6 = (
    obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar: number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // エラー。関数本体で obj に代入しているため絞り込まれない
    }
};
```

注: 条件式では、最大 5 段階の間接参照が解析されます。

型述語

TypeScript の型述語は、真偽値を返す関数で、変数の型をより具体的な型へ絞り込むために使用されます。

```
const isString = (value: unknown): value is string => typeof value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5 は、型述語（`x is T` など）を `.filter` などの関数内で自動的に推論します。そのため、undefined などの値が除外されたことを認識でき、型がより正確になってエラーが減ります。これは明確なチェック（例: `x !== undefined`）では機能しますが、`!!x` のような曖昧なチェックでは機能しません。

```
const nums = [1, null, 2].filter(x => x !== null);
```

判別可能なユニオン

TypeScript の判別可能なユニオンは、判別子と呼ばれる共通のプロパティを使用して、ユニオンで取り得る型の集合を絞り込むユニオン型の一種です。

```
type Square = {
    kind: 'square'; // 判別子
    size: number;
};

type Circle = {
    kind: 'circle'; // 判別子
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

never 型について

変数が値をまったく含められない型まで絞り込まれると、TypeScript コンパイラはその変数が never 型でなければならないと推論します。これは、never 型が決して生成されることのない値を表すためです。

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
        // val は string または number 以外になり得ないため、ここでは never
型
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

網羅性チェック

網羅性チェックは、判別可能なユニオンで取り得るすべてのケースが switch 文または if 文で処理されていることを保証する TypeScript の機能です。

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
```

```
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
            console.log(exhaustiveCheck); // この行は実行されない
    }
};
```

never 型は、default ケースが網羅的であることを保証し、Direction 型に新しい値が追加されたにもかかわらず switch 文で処理されていない場合に、TypeScript がエラーを発生させるために使用されます。

オブジェクト型

TypeScript のオブジェクト型は、オブジェクトの構造を記述します。オブジェクトのプロパティの名前と型に加えて、それらのプロパティが必須かオプショナルかを指定します。

TypeScript では、主に 2 つの方法でオブジェクト型を定義できます。

インターフェースは、プロパティの名前、型、オプショナルかどうかを指定して、オブジェクトの構造を定義します。

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

型エイリアスはインターフェースと同様に、オブジェクトの構造を定義します。ただし、既存の型または既存の型の組み合わせに基づ

く、新しいカスタム型を作成することもできます。これには、ユニオン型、交差型、その他の複雑な型の定義が含まれます。

```
type Point = {
    x: number;
    y: number;
};
```

型を匿名で定義することもできます。

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

タプル型（匿名）

タプル型は、固定された数の要素と、それぞれに対応する型を持つ配列を表す型です。タプル型は、特定の数の要素と、それぞれの型を固定された順序で強制します。タプル型は、特定の型を持つ値のコレクションを表し、配列内の各要素の位置に特定の意味がある場合に便利です。

```
type Point = [number, number];
```

名前付きタプル型（ラベル付き）

タプル型には、各要素にオプショナルなラベルまたは名前を含めることができます。これらのラベルは可読性とツールによる支援のためのものであり、タプルに対して実行できる操作には影響しません。

```
type T = string;
type Tuple1 = [T, T];
```

```
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // 名前付きタプルと匿名タプルの組み合わせ
```

固定長タプル

固定長タプルは、特定の型を持つ固定された数の要素を強制し、定義後にタプルの長さを変更できないようにする、特別な種類のタプルです。

固定長タプルは、特定の数の要素と特定の型を持つ値のコレクションを表す必要があり、タプルの長さと型が意図せず変更されないようにしたい場合に便利です。

```
const x = [10, 'hello'] as const;
x.push(2); // エラー
```

ユニオン型

ユニオン型は、複数の型のうちいずれか 1 つになり得る値を表す型です。ユニオン型は、取り得る各型の間に | 記号を使用して表します。

```
let x: string | number;
x = 'hello'; // 有効
x = 123; // 有効
```

交差型

交差型は、2 つ以上の型のすべてのプロパティを持つ値を表す型です。交差型は、各型の間に & 記号を使用して表します。

```
type X = {
    a: string;
```

```
};

type Y = {
    b: string;
};

type J = X & Y; // 交差

const j: J = {
    a: 'a',
    b: 'b',
};
```

型のインデックスアクセス

型のインデックスアクセスとは、明示的に宣言されていないプロパティの型を指定するインデックスシグネチャを使用して、事前には分からないキーでインデックスアクセスできる型を定義する機能を指します。

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // a を返す
```

値から型を取得する

TypeScript で値から型を取得するとは、型推論を通じて値または式から型が自動的に推論されることを指します。

```
const x = 'x'; // TypeScript は 'const'（不変）では 'x' を文字列リテラル型と推論するが、'let'（再代入可能）では 'string' に拡大する。
```

関数の戻り値から型を取得する

関数の戻り値から型を取得するとは、実装に基づいて関数の戻り値の型を自動的に推論する機能を指します。これにより TypeScript は、明示的な型アノテーションがなくても、関数が返す値の型を判断できます。

```
const add = (x: number, y: number) => x + y; // TypeScript は関数の戻り値の型を number と推論できる
```

モジュールから型を取得する

モジュールから型を取得するとは、モジュールがエクスポートした値を使用して、その型を自動的に推論する機能を指します。モジュールが特定の型を持つ値をエクスポートすると、TypeScript はその情報を使用して、別のモジュールへインポートされた際に、その値の型を自動的に推論できます。

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r は number
```

マップ型

TypeScript のマップ型を使用すると、マッピング関数で各プロパティを変換することにより、既存の型に基づいて新しい型を作成できます。既存の型をマッピングすることで、同じ情報を異なる形式で表す新しい型を作成できます。マップ型を作成するには、keyof 演算子を使用して既存の型のプロパティにアクセスし、それらを変更して新しい型を生成します。 次の例では、

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

MyMappedType が T のプロパティを順にマッピングし、各プロパティを元の型の配列とする新しい型を作成するよう定義しています。これを使用して MyNewType を作成し、MyType と同じ情報を表しつつ、各プロパティを配列にしています。

マップ型の修飾子

TypeScript のマップ型の修飾子を使用すると、既存の型内のプロパティを変換できます。

- `readonly` または `+readonly`: マップ型のプロパティを読み取り専用にします。
- `-readonly`: マップ型のプロパティを変更可能にします。
- `?`: マップ型のプロパティをオプショナルに指定します。

例:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // すべてのプロパティを読み取り専用にする
```

```
type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // すべてのプロパティを変更可能にする

type MyPartial<T> = { [P in keyof T]?: T[P] }; // すべてのプロパティをオプショナルにする
```

条件型

条件型は条件に依存する型を作成する方法で、作成する型は条件の結果に基づいて決まります。extends キーワードと三項演算子を使用して定義し、2 つの型から条件に応じて 1 つを選択します。

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // 型は true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // 型は false
```

分配条件型

分配条件型は、ユニオンの各メンバーに個別に変換を適用することで、型のユニオン全体に型を分配できる機能です。 これは、マップ型や高階型を扱う際に特に便利です。

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number | boolean | null
```

条件型における infer 型推論

infer キーワードは、条件型において、ジェネリック型パラメーターに依存する型から、そのパラメーターの型を推論（抽出）するために使用されます。これにより、より柔軟で再利用可能な型定義を記述できます。

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

定義済み条件型

TypeScript の定義済み条件型は、言語によって提供される組み込みの条件型です。指定された型の特性に基づいて、一般的な型変換を実行するよう設計されています。

`Exclude<UnionType, ExcludedType>`: Type から ExcludedType に代入可能なすべての型を除外します。

`Extract<Type, Union>`: Union から Type に代入可能なすべての型を抽出します。

`NonNullable<Type>`: Type から null と undefined を除外します。

`ReturnType<Type>`: 関数 Type の戻り値の型を抽出します。

`Parameters<Type>`: 関数 Type のパラメーター型を抽出します。

`Required<Type>`: Type のすべてのプロパティを必須にします。

`Partial<Type>`: Type のすべてのプロパティをオプショナルにします。

`Readonly<Type>`: Type のすべてのプロパティを読み取り専用にします。

テンプレートユニオン型

テンプレートユニオン型は、たとえば次のように、型システム内でテキストを結合および操作するために使用できます。

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type  ProductId  =  `id-${Products}-${Status}`;  // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

any 型

any 型は、あらゆる種類の値（プリミティブ、オブジェクト、配列、関数、エラー、シンボル）を表すために使用できる特殊な型（普遍上位型）です。コンパイル時に値の型が不明な場合や、TypeScript の型定義がない外部 API またはライブラリの値を扱う場合によく使用されます。

any 型を使用することで、値を一切制約なく扱うべきであることを TypeScript コンパイラに示します。コードの型安全性を最大限に高めるため、次の点を検討してください。

- any の使用は、型が本当に不明な特定のケースに限定します。
- 関数から any 型を返さないでください。その関数を使用するコードの型安全性が低下します。
- コンパイラによるチェックを抑制する必要がある場合は、any の代わりに @ts-ignore を使用します。

```
let value: any;
value = true; // 有効
value = 7; // 有効
```

unknown 型

TypeScript の unknown 型は、型が不明な値を表します。あらゆる型の値を許可する any 型とは異なり、unknown を特定の方法で使用する前には型チェックまたはアサーションが必要です。そのため、より具体的な型を最初にアサーションするか絞り込まない限り、unknown に対する操作は許可されません。

unknown 型は、any と unknown 自体にのみ代入可能であり、any に代わる型安全な選択肢です。

```
let value: unknown;

let value1: unknown = value; // 有効
let value2: any = value; // 有効
let value3: boolean = value; // 無効
let value4: number = value; // 無効

const add = (a: unknown, b: unknown): number | undefined =>
    typeof a === 'number' && typeof b === 'number' ? a + b : undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

void 型

void 型は、関数が値を返さないことを示すために使用されます。

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

never 型

never 型は、決して発生しない値を表します。決して戻らない関数や式、またはエラーをスローする関数や式を表すために使用されます。

たとえば、無限ループは次のようになります。

```
const infiniteLoop = (): never => {
    while (true) {
        // 何らかの処理を行う
    }
};
```

エラーをスローする場合は、次のようになります。

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

never 型は、型安全性を確保し、コード内の潜在的なエラーを検出する際に便利です。ほかの型や制御フロー文と組み合わせて使用すると、TypeScript がより正確な型を解析および推論するのに役立ちます。例:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // 上に移動
            break;
        case 'down':
            // 下に移動
            break;
        default:
            const exhaustiveCheck: never = direction;
            throw new Error(`Unhandled direction: ${exhaustiveCheck}`);
```

```
    }
};
```

インターフェースと型

共通の構文

TypeScript では、インターフェースはオブジェクトの構造を定義し、オブジェクトが持つ必要のあるプロパティまたはメソッドの名前と型を指定します。TypeScript でインターフェースを定義する一般的な構文は次のとおりです。

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

型定義の場合も同様です。

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` または `type TypeName`: インターフェースの名前を定義します。 `property1: Type1`: インターフェースのプロパティと、それに対応する型を指定します。複数のプロパティを定義でき、それぞれをセミコロンで区切ります。 `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: インターフェースのメソッドを指定します。メ

ソッドは、名前、その後に丸括弧内のパラメーターリスト、そして戻り値の型という順序で定義します。複数のメソッドを定義でき、それぞれをセミコロンで区切ります。

インターフェースの例:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

型の例:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

TypeScript では、型を使用してデータの構造を定義し、型チェックを強制します。TypeScript で型を定義するための一般的な構文は、具体的なユースケースに応じていくつかあります。例を次に示します。

基本型

```
let myNumber: number = 123; // number 型
let myBoolean: boolean = true; // boolean 型
let myArray: string[] = ['a', 'b']; // 文字列の配列
let myTuple: [string, number] = ['a', 123]; // タプル
```

オブジェクトとインターフェース

```
const x: { name: string; age: number } = { name: 'Simon', age: 7 };
```

ユニオン型と交差型

```
type MyType = string | number; // ユニオン型
let myUnion: MyType = 'hello'; // 文字列にできる
myUnion = 123; // または数値にできる

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // 交差型
let myCombined: CombinedType = { name: 'John', age: 25 }; // name と age の両方のプロパティを持つオブジェクト
```

組み込みのプリミティブ型

TypeScript には、変数、関数パラメーター、戻り値の型を定義するために使用できる、いくつかの組み込みプリミティブ型があります。

- number: 整数や浮動小数点数を含む数値を表します。
- string: テキストデータを表します。
- boolean: true または false のいずれかになる論理値を表します。
- null: 値が存在しないことを表します。
- undefined: 値が代入されていない、または定義されていないことを表します。
- symbol: 一意の識別子を表します。シンボルは通常、オブジェクトのプロパティのキーとして使用されます。
- bigint: 任意精度の整数を表します。
- any: 動的または不明な型を表します。any 型の変数には任意の型の値を格納でき、型チェックを回避します。
- void: 型が存在しないことを表します。一般に、値を返さない関数の戻り値の型として使用されます。

- never: 決して発生しない値の型を表します。通常、エラーをスローする関数や無限ループに入る関数の戻り値の型として使用されます。

一般的な組み込み JavaScript オブジェクト

TypeScript は JavaScript のスーパーセットであり、一般的に使用されるすべての組み込み JavaScript オブジェクトが含まれます。これらのオブジェクトの詳細な一覧は、Mozilla Developer Network（MDN）のドキュメントサイトで確認できます。https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

一般的に使用される組み込み JavaScript オブジェクトの一部を次に示します。

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise

- Intl

オーバーロード

TypeScript の関数オーバーロードを使用すると、1 つの関数名に対して複数の関数シグネチャを定義でき、複数の方法で呼び出せる関数を定義できます。例を次に示します。

```
// オーバーロード
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// 実装
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // 有効
sayHi(['aa', 'bb']); // 有効
```

class 内で関数オーバーロードを使用する別の例を次に示します。

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // オーバーロード
```

```
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // 実装
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
            return name.map(name => `${this.message}, ${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

マージと拡張

マージと拡張は、型とインターフェースを扱う際の 2 つの異なる概念を指します。

マージを使用すると、同じ名前の複数の宣言を 1 つの定義にまとめられます。たとえば、同じ名前のインターフェースを複数回定義した場合です。

```
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
```

```
    b: 7,
};
```

拡張とは、既存の型またはインターフェースを拡張または継承して、新しい型を作成する機能を指します。元の定義を変更せずに、既存の型へ追加のプロパティまたはメソッドを加える仕組みです。例:

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

型とインターフェースの違い

宣言のマージ（拡張）:

インターフェースは宣言のマージをサポートしています。つまり、同じ名前のインターフェースを複数定義すると、TypeScript はそれらを、プロパティとメソッドを組み合わせた単一のインターフェー

スにマージします。一方、型は宣言のマージをサポートしていません。これは、元の定義を変更したり、不足している型や誤った型にパッチを当てたりせずに、追加の機能を加えたい場合や既存の型をカスタマイズしたい場合に役立ちます。

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

他の型やインターフェースの拡張:

型とインターフェースはどちらも他の型やインターフェースを拡張できますが、構文が異なります。インターフェースでは、extends キーワードを使用して他のインターフェースからプロパティとメソッドを継承します。ただし、インターフェースはユニオン型のような複雑な型を拡張できません。

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
```

```
    z: 'z',
};
```

型では、& 演算子を使用して複数の型を単一の型（交差型）に結合します。

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

ユニオン型と交差型:

ユニオン型と交差型を定義する場合、型のほうが柔軟です。type キーワードを使用すると、| 演算子でユニオン型を、& 演算子で交差型を簡単に作成できます。インターフェースでも間接的にユニオン型を表現できますが、交差型の組み込みサポートはありません。

```
type Department = 'dep-x' | 'dep-y'; // ユニオン

type Person = {
    name: string;
    age: number;
};
```

```
type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // 交差
```

インターフェースを使用した例:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // インターフェースのユニオン
```

クラス

クラスの一般的な構文

TypeScript では、クラスを定義するために class キーワードを使用します。以下に例を示します。

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
            `Hello, my name is ${this.name} and I am ${this.age} years old.`
```

```
        );
    }
}
```

class キーワードは、「Person」という名前のクラスを定義するために使用されています。

このクラスには、string 型の name と number 型の age という 2 つのプライベートプロパティがあります。

コンストラクターは constructor キーワードを使用して定義されています。name と age をパラメーターとして受け取り、対応するプロパティに代入します。

このクラスには、挨拶メッセージをログに出力する sayHi という名前の public メソッドがあります。

TypeScript でクラスのインスタンスを作成するには、new キーワードの後にクラス名と丸括弧 () を続けます。例:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // 出力: Hello, my name is John Doe and I am 25 years old.
```

コンストラクター

コンストラクターはクラス内の特別なメソッドで、クラスのインスタンスが作成されるときにオブジェクトのプロパティを初期化するために使用されます。

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
```

```
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age} years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

次の構文を使用してコンストラクターをオーバーロードできます。

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

TypeScript では複数のコンストラクターオーバーロードを定義できますが、実装は 1 つだけで、すべてのオーバーロードと互換性がなければなりません。これはオプショナルパラメーターを使用して実現できます。

```
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

プライベートコンストラクターとプロテクテッドコンストラクター

TypeScript では、コンストラクターを private または protected としてマークでき、これによりアクセスできる範囲と使用方法が制限されます。

プライベートコンストラクター: クラス自体の内部からのみ呼び出せます。プライベートコンストラクターは、シングルトンパターンを適用したい場合や、インスタンスの作成をクラス内のファクトリーメソッドに制限したい場合によく使用されます。

プロテクテッドコンストラクター: 直接インスタンス化すべきではないものの、サブクラスから拡張できる基底クラスを作成したい場合に便利です。

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// 基底クラスを直接インスタンス化しようとするとエラーになる
// const baseObj = new BaseClass(); // エラー: クラス 'BaseClass' のコンストラクターは protected です。

// 派生クラスのインスタンスを作成
const derivedObj = new DerivedClass(10);
```

アクセス修飾子

アクセス修飾子 private、protected、public は、TypeScript クラスのプロパティやメソッドなどのクラスメンバーの可視性とアクセスできる範囲を制御するために使用されます。これらの修飾子は、カプセル化を適用し、クラスの内部状態へのアクセスや変更に境界を設けるうえで不可欠です。

private 修飾子は、クラスメンバーへのアクセスを、それを含むクラス内のみに制限します。

protected 修飾子は、クラスメンバーへのアクセスを、それを含むクラスとその派生クラス内で許可します。

public 修飾子はクラスメンバーへの無制限のアクセスを提供し、どこからでもアクセスできるようにします。

ゲッターとセッター

ゲッターとセッターは、クラスプロパティへのアクセスと変更の動作を独自に定義できる特別なメソッドです。オブジェクトの内部状態をカプセル化し、プロパティの値を取得または設定するときに追加のロジックを実行できます。 TypeScript では、ゲッターとセッターはそれぞれ get キーワードと set キーワードを使用して定義します。次に例を示します。

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
```

```
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

クラスの自動アクセサー

TypeScript バージョン 4.9 では、今後の ECMAScript 機能である自動アクセサーのサポートが追加されています。自動アクセサーはクラスプロパティに似ていますが、「accessor」キーワードを使用して宣言します。

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

自動アクセサーは、アクセス不能なプロパティを操作するプライベートな get アクセサーと set アクセサーに展開されます。

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
```

```
    }
}
```

this

TypeScript では、this キーワードはメソッドまたはコンストラクター内におけるクラスの現在のインスタンスを指します。これにより、クラス自体のスコープ内から、そのクラスのプロパティとメソッドにアクセスして変更できます。 これは、オブジェクト自身のメソッド内で、その内部状態にアクセスして操作する方法を提供します。

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

パラメータープロパティ

パラメータープロパティを使用すると、コンストラクターのパラメーター内でクラスプロパティを直接宣言して初期化でき、定型的なコードを省けます。例:

```
class Person {
    constructor(
        private name: string,
```

```
        public age: number
    ) {
        // コンストラクター内の "private" と "public" キーワードは
        // 対応するクラスプロパティを自動的に宣言して初期化する。
    }
    public introduce(): void {
        console.log(
            `Hello, my name is ${this.name} and I am ${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

抽象クラス

TypeScript では、抽象クラスは主に継承に使用されます。サブクラスが継承できる共通のプロパティとメソッドを定義する方法を提供します。 これは、共通の動作を定義し、サブクラスが特定のメソッドを実装することを強制したい場合に便利です。抽象基底クラスがサブクラスに共有インターフェースと共通機能を提供するクラス階層を作成できます。

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
```

```
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // 出力: Whiskers meows.
```

ジェネリクスを使用するクラス

ジェネリクスを使用するクラスでは、さまざまな型を扱える再利用可能なクラスを定義できます。

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

デコレーター

デコレーターは、メタデータの追加、動作の変更、検証、または対象要素の機能の拡張を行う仕組みを提供します。デコレーターは実行時に実行される関数です。1 つの宣言に複数のデコレーターを適用できます。

デコレーターは実験的な機能であり、以下の例は ES6 を使用する TypeScript バージョン 5 以降でのみ互換性があります。

TypeScript 5 より前のバージョンでは、`experimentalDecorators` プロパティを `tsconfig.json` で使用するか、コマンドラインで `--experimentalDecorators` を使用して有効にする必要があります（ただし、以下の例は動作しません）。

デコレーターの一般的なユースケースには、次のようなものがあります。

- プロパティの変更の監視。
- メソッド呼び出しの監視。
- 追加のプロパティやメソッドの追加。
- 実行時の検証。
- 自動的なシリアライズとデシリアライズ。
- ロギング。
- 認可と認証。
- エラーの防止。

注: バージョン 5 のデコレーターでは、パラメーターをデコレートできません。

デコレーターの種類:

クラスデコレーター

クラスデコレーターは、プロパティやメソッドを追加したり、クラスのインスタンスを収集したりするなど、既存のクラスを拡張する場合に便利です。次の例では、クラスを文字列表現に変換する toString メソッドを追加します。

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}

@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
```

```
const person = new Person('Simon');
/* ログ出力:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

プロパティデコレーター

プロパティデコレーターは、初期値を変更するなど、プロパティの動作を変更する場合に便利です。次のコードは、プロパティが常に大文字になるように設定するスクリプトです。

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // ログ出力: HELLO!
```

メソッドデコレーター

メソッドデコレーターを使用すると、メソッドの動作を変更または拡張できます。以下は、単純なロガーの例です。

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

    function replacementMethod(this: This, ...args: Args): Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}

new MyClass().sayHello();
```

次の内容がログに出力されます。

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

ゲッターとセッターのデコレーター

ゲッターとセッターのデコレーターを使用すると、クラスアクセサーの動作を変更または拡張できます。たとえば、プロパティへの代入を検証する場合に便利です。次に、ゲッターデコレーターの簡単な例を示します。

```
function range<This, Return extends number>(min: number, max: number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}
```

```
const obj = new MyClass(10);
console.log(obj.getValue); // 有効: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // スロー: Invalid!
```

デコレーターメタデータ

デコレーターメタデータにより、デコレーターが任意のクラスにメタデータを適用し、利用する処理が簡単になります。デコレーターはコンテキストオブジェクトの新しい metadata プロパティにアクセスでき、このプロパティはプリミティブとオブジェクトの両方に対するキーとして使用できます。 メタデータ情報には、クラスの Symbol.metadata を介してアクセスできます。

メタデータは、デバッグ、シリアライズ、デコレーターを用いた依存性注入など、さまざまな目的に使用できます。

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // 簡単なポリフィル

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
    | ClassMethodDecoratorContext; // コンテキストはプロパティ metadata を含
む: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // メタデータオブジェクトにプリミティブ値を設定
    context.metadata[context.name] = true;
}

class MyClass {
```

```
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // メタデータ情報を取得

console.log(JSON.stringify(metadata)); // {"bar":true,"baz":true,"foo":true}
```

継承

継承とは、あるクラスが基底クラスまたはスーパークラスと呼ばれる別のクラスからプロパティとメソッドを継承できる仕組みを指します。派生クラスは子クラスまたはサブクラスとも呼ばれ、新しいプロパティやメソッドを追加したり、既存のものをオーバーライドしたりすることで、基底クラスの機能を拡張して特化できます。

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
```

```
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// 基底クラスのインスタンスを作成
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// 派生クラスのインスタンスを作成
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript は従来の意味での多重継承をサポートしておらず、代わりに単一の基底クラスからの継承を許可します。 TypeScript は複数のインターフェースをサポートしています。インターフェースはオブジェクトの構造に対する契約を定義でき、クラスは複数のインターフェースを実装できます。これにより、クラスは複数のソースから動作と構造を継承できます。

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}
```

```
class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

TypeScript の class キーワードは、JavaScript と同様に、糖衣構文と呼ばれることがよくあります。これは、クラスベースの方法でオブジェクトを作成して操作するための、より馴染みのある構文を提供する目的で ECMAScript 2015（ES6）に導入されました。ただし、TypeScript は JavaScript のスーパーセットであるため、最終的には JavaScript にコンパイルされ、その中核は引き続きプロトタイプベースである点に注意が必要です。

静的メンバー

TypeScript には静的メンバーがあります。クラスの静的メンバーにアクセスするには、オブジェクトを作成せずに、クラス名の後にドットを続けて使用できます。

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
```

```
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

プロパティの初期化

TypeScript でクラスのプロパティを初期化する方法はいくつかあります。

インライン:

次の例では、クラスのインスタンスが作成されるときに、これらの初期値が使用されます。

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

コンストラクター内:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

コンストラクターパラメーターを使用する方法:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
        // プロパティに値を明示的に代入する必要はない。
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

メソッドのオーバーロード

メソッドのオーバーロードにより、クラスは同じ名前で、パラメーターの型またはパラメーター数が異なる複数のメソッドを持つことができます。これにより、渡された引数に応じて異なる方法でメソッドを呼び出せます。

```
class MyClass {
    add(a: number, b: number): number; // オーバーロードシグネチャ 1
    add(a: string, b: string): string; // オーバーロードシグネチャ 2

    add(a: number | string, b: number | string): number | string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}
```

```
const r = new MyClass();
console.log(r.add(10, 5)); // ログ出力 15
```

ジェネリクス

ジェネリクスを使用すると、複数の型を扱える再利用可能なコンポーネントや関数を作成できます。ジェネリクスでは、型、関数、インターフェースを型パラメーター化できるため、事前に明示的に指定することなく、さまざまな型を扱えるようになります。

ジェネリクスを使用すると、コードの柔軟性と再利用性を高められます。

ジェネリック型

ジェネリック型を定義するには、山括弧（<>）を使用して型パラメーターを指定します。例:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

ジェネリッククラス

ジェネリクスはクラスにも適用でき、型パラメーターを使用することで複数の型を扱えます。これは、型安全性を維持しながらさまざ

まなデータ型を扱える、再利用可能なクラス定義を作成する場合に便利です。

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

ジェネリック制約

ジェネリック型パラメーターは、extends キーワードの後に、その型パラメーターが満たす必要のある型またはインターフェースを続けることで制約できます。

次の例では、有効であるために T が適切に型付けされた length プロパティを持っている必要があります。

```
const printLen = <T extends { length: number }>(value: T): void => {
    console.log(value.length);
};

printLen('Hello'); // 5
```

```
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // 無効
```

バージョン 3.4 RC で導入された注目すべきジェネリック機能の 1 つに、ジェネリック型引数を伝播する高階関数の型推論があります。

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

この機能により、関数型プログラミングで一般的な、型安全なポイントフリースタイルのプログラミングが容易になります。

ジェネリクスのコンテキストに基づく絞り込み

ジェネリクスのコンテキストに基づく絞り込みとは、TypeScript において、ジェネリック型パラメーターが使用されるコンテキストに基づいて、コンパイラがその型を絞り込める仕組みです。条件文でジェネリック型を扱う場合に便利です。

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // 値は型 'string' に絞り込まれる
        console.log(value.length);
    } else if (typeof value === 'number') {
```

```
        // 値は型 'number' に絞り込まれる
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

消去される構造的型

TypeScript では、オブジェクトが特定の厳密な型と一致する必要はありません。たとえば、あるインターフェースの要件を満たすオブジェクトを作成した場合、それらの間に明示的な関連付けがなくても、そのインターフェースが必要な場所でそのオブジェクトを使用できます。 例:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // 有効
```

名前空間

TypeScript では、名前空間はコードを論理的なコンテナーに整理するために使用され、名前の衝突を防ぎ、関連するコードをまとめる方法を提供します。 `export` キーワードを使用すると、モジュールの外部から名前空間にアクセスできます。

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

シンボル

シンボルは、プログラムの存続期間全体にわたってグローバルに一意であることが保証された不変の値を表すプリミティブデータ型です。

シンボルはオブジェクトプロパティのキーとして使用でき、列挙されないプロパティを作成する方法を提供します。

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};
```

```
console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

WeakMap と WeakSet では、シンボルをキーとして使用できるようになりました。

トリプルスラッシュディレクティブ

トリプルスラッシュディレクティブは、ファイルの処理方法についてコンパイラに指示を与える特別なコメントです。これらのディレクティブは 3 つの連続するスラッシュ（///）で始まり、通常は TypeScript ファイルの先頭に配置され、実行時の動作には影響しません。

トリプルスラッシュディレクティブは、外部依存関係の参照、モジュール読み込み動作の指定、特定のコンパイラ機能の有効化または無効化などに使用されます。いくつか例を示します。

宣言ファイルの参照:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

モジュール形式の指定:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

コンパイラオプションの有効化。次の例では strict モード:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

型の操作

型から型を作成する

既存の型を合成、操作、変換することで、新しい型を作成できます。

交差型（&）：

複数の型を単一の型に結合できます。

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // A と B の交差
const obj: C = { foo: 42, bar: 'hello' };
```

ユニオン型（|）：

複数の型のうちいずれか 1 つになり得る型を定義できます。

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

マップ型:

既存の型のプロパティを変換して、新しい型を作成できます。

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // プロパティが読み取り専用になる
```

条件型:

何らかの条件に基づいて型を作成できます。

```
type ExtractParam<T> = T extends (param: infer P) => any ? P : never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

インデックスアクセス型

TypeScript では、インデックス `Type[Key]` を使用して、別の型内のプロパティの型にアクセスし、操作できます。

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

ユーティリティ型

型の操作に使用できる組み込みユーティリティ型がいくつかあります。以下は、最もよく使用されるものの一覧です。

Awaited<T>

Promise 型を再帰的にアンラップする型を構築します。

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

T のすべてのプロパティをオプショナルに設定した型を構築します。

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?: number | undefined; }
```

Required<T>

T のすべてのプロパティを必須に設定した型を構築します。

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

T のすべてのプロパティを読み取り専用に設定した型を構築します。

```
type Person = {
    name: string;
    age: number;
};
```

```ts
type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // 無効
```

Record<K, T>

K をキーとし、T を値の型とするプロパティの集合を持つ型を構築します。

```ts
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

T から指定されたプロパティ K を選択して型を構築します。

```ts
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

T から指定されたプロパティ K を除外して型を構築します。

```
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

T から U 型のすべての値を除外して型を構築します。

```
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

T から U 型のすべての値を抽出して型を構築します。

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

T から null と undefined を除外して型を構築します。

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

関数型 T のパラメーター型を抽出します。

```ts
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

コンストラクター関数型 T のパラメーター型を抽出します。

```ts
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

関数型 T の戻り値の型を抽出します。

```ts
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

クラス型 T のインスタンス型を抽出します。

```ts
class Person {
    name: string;

    constructor(name: string) {
```

```
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

関数型 T から 'this' パラメーターの型を抽出します。

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

OmitThisParameter<T>

関数型 T から 'this' パラメーターを削除します。

```
function capitalize(this: String) {
    return this[0].toUpperCase() + this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

コンテキストに基づく this 型のマーカーとして機能します。

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } & ThisType<Logger> = {
    hello: function () {
        this.log('some error'); // 有効。"log" は "this" の一部であるため。
        this.update(); // 無効
    },
};
```

Uppercase<T>

入力型 T の名前を大文字にします。

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

入力型 T の名前を小文字にします。

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

入力型 T の名前の先頭を大文字にします。

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

入力型 T の名前の先頭を小文字にします。

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer は、ジェネリック関数のスコープ内で型が自動推論されるのを阻止するために設計されたユーティリティ型です。

例:

```
// ジェネリック関数のスコープ内での型の自動推論。
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // ここでの型は ("a" | "b" | "c")[]
```

NoInfer を使用する場合:

```
// NoInfer を使用して型推論を防ぐ関数の例
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // エラー: 型 '"c"' の引数を型 '"a" | "b"' のパラメーターに割り当てることはできません。
```

その他

エラーと例外処理

TypeScript では、標準の JavaScript エラー処理機構を使用してエラーをキャッチし、処理できます。

Try-Catch-Finally ブロック:

```
try {
    // エラーをスローする可能性のあるコード
} catch (error) {
    // エラーを処理
} finally {
    // 常に実行されるコード。finally は省略可能
}
```

異なる種類のエラーを処理することもできます。

```
try {
    // さまざまな型のエラーをスローする可能性のあるコード
} catch (error) {
    if (error instanceof TypeError) {
        // TypeError を処理
    } else if (error instanceof RangeError) {
        // RangeError を処理
    } else {
        // その他のエラーを処理
    }
}
```

カスタムエラー型:

Error クラスを拡張することで、より具体的なエラーを指定できます。

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
```

```
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

ミックスインクラス

ミックスインクラスを使用すると、複数のクラスの振る舞いを組み合わせて構成し、1つのクラスにまとめることができます。深い継承チェーンを必要とせずに、機能を再利用して拡張する方法を提供します。

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// MyClass を拡張し、Identifiable と Selectable の振る舞いを含める
```

```
interface MyClass extends Identifiable, Selectable {}

// クラスにミックスインを適用する関数
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {
        Object.getOwnPropertyNames(baseCtor.prototype).forEach(name => {
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                Object.defineProperty(source.prototype, name, descriptor);
            }
        });
    });
}

// MyClass にミックスインを適用
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

非同期言語機能

TypeScript は JavaScript のスーパーセットであるため、次のような JavaScript の非同期言語機能が組み込まれています。

Promise：

Promise は非同期処理とその結果を扱うための仕組みで、`.then()` や `.catch()` などのメソッドを使用して成功時とエラー時の状態を処理します。

詳細はこちら：https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await：

Async/await キーワードを使用すると、Promise を扱う際に、より同期処理に近い見た目の構文を記述できます。`async` キーワードは非同期関数を定義するために使用され、`await` キーワードは async 関数内で、Promise が解決または拒否されるまで実行を一時停止するために使用されます。

詳細はこちら：https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

次の API は TypeScript で十分にサポートされています。

Fetch API：https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers：https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers：https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket：https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

イテレーターとジェネレーター

TypeScript はイテレーターとジェネレーターの両方を十分にサポートしています。

イテレーターはイテレータープロトコルを実装するオブジェクトであり、コレクションやシーケンスの要素に 1 つずつアクセスする方法を提供します。反復処理における次の要素へのポインターを含む構造です。イテレーターには next() メソッドがあり、シーケンス内の次の値と、シーケンスが done であるかどうかを示す真偽値を返します。

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}
```

```
const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

ジェネレーターは、イテレーターの作成を簡素化する function* 構文を使用して定義される特殊な関数です。yield キーワードを使用して値のシーケンスを定義し、値が要求されると実行を自動的に一時停止して再開します。

ジェネレーターを使用するとイテレーターを簡単に作成でき、特に大規模または無限のシーケンスを扱う場合に役立ちます。

例：

```
function* numberGenerator(start: number, end: number): Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript は非同期イテレーターと非同期ジェネレーターもサポートしています。

詳細はこちら：

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

TsDocs・JSDoc リファレンス

JavaScript のコードベースを扱う場合、型情報を提供する追加のアノテーションを含む JSDoc コメントを使用することで、TypeScript が適切な型を推論できるように支援できます。

例：

```
/**
 * 指定した数値の累乗を計算
 * @constructor
 * @param {number} base - 式の底の値
 * @param {number} exponent - 式の指数の値
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

完全なドキュメントはこちらのリンクで提供されています：https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

バージョン 3.7 以降では、JavaScript の JSDoc 構文から .d.ts 型定義を生成できます。詳細はこちら：https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

@types 組織配下のパッケージには、既存の JavaScript ライブラリやモジュールに型定義を提供するための、特別な命名規則が使われています。たとえば、次を使用すると、

```
npm install --save-dev @types/lodash
```

現在のプロジェクトに `lodash` の型定義がインストールされます。

`@types` パッケージの型定義に貢献するには、https://github.com/DefinitelyTyped/DefinitelyTyped にプルリクエストを送信してください。

JSX

JSX（JavaScript XML）は JavaScript 言語の構文を拡張するもので、JavaScript または TypeScript ファイル内に HTML に似たコードを記述できます。React で HTML 構造を定義するためによく使用されます。

TypeScript は型チェックと静的解析を提供することで JSX の機能を拡張します。

JSX を使用するには、`tsconfig.json` ファイルで `jsx` コンパイラオプションを設定する必要があります。一般的な設定オプションは次の 2 つです。

- “preserve”：JSX を変更せずに .jsx ファイルを出力します。このオプションは、JSX 構文をそのまま保持し、コンパイル処理中に変換しないよう TypeScript に指示します。変換を処理す

る Babel などの別のツールがある場合に、このオプションを使用できます。
- “react”：TypeScript に組み込まれた JSX 変換を有効にします。React.createElement が使用されます。

すべてのオプションはこちらで確認できます：https://www.typescriptlang.org/tsconfig#jsx

ES6 モジュール

TypeScript は ES6（ECMAScript 2015）およびそれ以降の多くのバージョンをサポートしています。つまり、アロー関数、テンプレートリテラル、クラス、モジュール、分割代入などの ES6 構文を使用できます。

プロジェクトで ES6 機能を有効にするには、tsconfig.json で target プロパティを指定できます。

設定例：

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

ES7 べき乗演算子

べき乗（**）演算子は、第 1 オペランドを第 2 オペランドの累乗にした値を計算します。Math.pow() と同様に機能しますが、オペランドとして BigInt も受け入れられます。 TypeScript では、tsconfig.json ファイルの target を es2016 以上に設定することで、この演算子が完全にサポートされます。

```
console.log(2 ** (2 ** 2)); // 16
```

for-await-of 文

これは TypeScript で完全にサポートされている JavaScript の機能で、ターゲットバージョンを es2018 にすると、非同期反復可能オブジェクトを反復処理できます。

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

new target メタプロパティ

TypeScript では new.target メタプロパティを使用できます。これにより、関数またはコンストラクターが new 演算子を使用して呼び出されたかどうかを判定できます。コンストラクター呼び出しの結果としてオブジェクトが作成されたかどうかを検出できます。

```
class Parent {
    constructor() {
        console.log(new.target); // インスタンスの作成に使用されたコンストラクター関数をログ出力
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

動的 import 式

TypeScript でサポートされている動的 import の ECMAScript 提案を使用して、条件に応じてモジュールを読み込んだり、必要に応じて遅延読み込みしたりできます。

TypeScript における動的 import 式の構文は次のとおりです。

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
        const widget = await import('./widget'); // 動的インポート
        widget.render(container);
    }
}

renderWidget();
```

“tsc –watch”

このコマンドは --watch パラメーターを指定して TypeScript コンパイラを起動し、TypeScript ファイルが変更されるたびに自動的に再コンパイルできるようにします。

```
tsc --watch
```

TypeScript バージョン 4.9 以降、ファイル監視は主にファイルシステムイベントに依存し、イベントベースのウォッチャーを確立できない場合は自動的にポーリングへ切り替わります。

非 null アサーション演算子

非 null アサーション演算子（後置の !）は、確実な代入アサーションとも呼ばれ、TypeScript の静的型解析が null または undefined の可能性を示している場合でも、変数やプロパティが null または undefined ではないと表明できる TypeScript の機能です。この機能を使用すると、明示的なチェックをすべて取り除くことができます。

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

デフォルト値を持つ宣言

デフォルト値を持つ宣言は、変数またはパラメーターにデフォルト値が代入される場合に使用されます。つまり、その変数またはパラメーターに値が指定されていない場合、デフォルト値が使用されます。

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

オプショナルチェーン

オプショナルチェーン演算子 `?.` は、プロパティやメソッドにアクセスする際、通常のドット演算子（`.`）と同様に機能します。ただし、エラーをスローする代わりに式を終了して `undefined` を返すことで、null または undefined の値を適切に処理します。

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

null 合体演算子

null 合体演算子 `??` は、左辺の値が `null` または `undefined` の場合は右辺の値を返し、それ以外の場合は左辺の値を返します。

```
const foo = null ?? 'foo';
console.log(foo); // foo
```

```
const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

テンプレートリテラル型

テンプレートリテラル型を使用すると、型レベルで文字列値を操作し、既存の文字列型に基づいて新しい文字列型を生成できます。文字列ベースの操作から、より表現力が高く正確な型を作成するのに役立ちます。

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

関数オーバーロード

関数オーバーロードを使用すると、同じ関数名に対して、パラメーター型と戻り値の型がそれぞれ異なる複数の関数シグネチャを定義できます。 オーバーロードされた関数を呼び出すと、TypeScriptは指定された引数を使用して正しい関数シグネチャを判定します。

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
```

```
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

再帰型

再帰型とは、それ自身を参照できる型です。これは、連結リスト、木、グラフなど、階層構造または再帰構造（無限にネストする可能性がある）を持つデータ構造を定義するのに役立ちます。

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

再帰条件型

TypeScript では、ロジックと再帰を使用して複雑な型の関係を定義できます。 簡単な言葉で、順に説明します。

条件型を使用すると、真偽条件に基づいて型を定義できます。

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

再帰とは、型定義の中でその型自体を参照する型定義を意味します。

```
type Json = string | number | boolean | null | Json[] | { [key: string]:
Json };

const data: Json = {
```

```
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

再帰条件型は、条件ロジックと再帰の両方を組み合わせます。これは、型定義が条件ロジックを通じてそれ自身に依存できることを意味し、複雑で柔軟な型の関係を作成します。

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 | 1 | 6
```

Node における ECMAScript モジュールのサポート

Node.js はバージョン 15.3.0 から ECMAScript モジュールのサポートを追加し、TypeScript はバージョン 4.7 以降、Node.js 向けの ECMAScript モジュールをサポートしています。このサポートは、tsconfig.json ファイルで `module` プロパティに値 `nodenext` を使用することで有効にできます。例を示します。

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js はモジュール用に 2 つのファイル拡張子をサポートしています。ES モジュールには `.mjs`、CommonJS モジュールには `.cjs` を

使用します。TypeScript で対応するファイル拡張子は、ES モジュールには .mts、CommonJS モジュールには .cts です。TypeScript コンパイラがこれらのファイルを JavaScript にトランスパイルすると、.mjs ファイルと .cjs ファイルが作成されます。

プロジェクトで ES モジュールを使用する場合は、package.json ファイルで type プロパティを “module” に設定できます。これにより、プロジェクトを ES モジュールプロジェクトとして扱うよう Node.js に指示します。

さらに、TypeScript は .d.ts ファイルでの型宣言もサポートしています。これらの宣言ファイルは、TypeScript で記述されたライブラリやモジュールの型情報を提供し、他の開発者が TypeScript の型チェック機能と自動補完機能を利用できるようにします。

アサーション関数

TypeScript におけるアサーション関数は、戻り値に基づいて特定の条件の検証を示す関数です。最も単純な形式では、assert 関数は指定された述語を調べ、述語が false と評価された場合にエラーを発生させます。

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

また、関数式として宣言することもできます。

```
type AssertIsNumber = (value: unknown) => asserts value is number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
```

```
        throw new Error('Not a number');
    }
};
```

アサーション関数には型ガードとの類似点があります。型ガードはもともと、実行時チェックを行い、特定のスコープ内で値の型を保証するために導入されました。 具体的には、型ガードは型述語を評価し、その述語が true か false かを示す真偽値を返す関数です。これはアサーション関数とは少し異なります。アサーション関数は、述語が満たされない場合に false を返すのではなく、エラーをスローすることを目的としています。

型ガードの例：

```
const isNumber = (value: unknown): value is number => typeof value ===
'number';
```

可変長タプル型

可変長タプル型は TypeScript バージョン 4.0 で導入された機能です。まず、タプルとは何かを振り返ることから始めましょう。

タプル型は、長さが定義され、各要素の型が既知である配列です。

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

「variadic」という用語は、引数の個数が不定であること（可変数の引数を受け入れること）を意味します。

可変長タプルは、これまでと同じすべてのプロパティを持ちながら、正確な形状がまだ定義されていないタプル型です。

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

前のコードでは、渡されたジェネリック型引数 T によってタプルの形状が定義されていることが分かります。

可変長タプルでは複数の型パラメーターを使用できるため、非常に柔軟です。

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

新しい可変長タプルでは、次の機能を使用できます。

- タプル型構文内のスプレッドをジェネリックにできるようになったため、操作対象となる実際の型が分からない場合でも、タプルと配列に対する高階操作を表現できます。
- rest 要素をタプル内の任意の位置に配置できます。

例：

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}
```

```
concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

ボックス化された型

ボックス化された型とは、プリミティブ型をオブジェクトとして表現するために使用されるラッパーオブジェクトを指します。これらのラッパーオブジェクトは、プリミティブ値で直接利用できない追加の機能やメソッドを提供します。

string プリミティブで charAt や normalize のようなメソッドにアクセスすると、JavaScript はそれを String オブジェクトでラップし、メソッドを呼び出してから、そのオブジェクトを破棄します。

デモ：

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript は、プリミティブとそれに対応するオブジェクトラッパーに個別の型を提供することで、この違いを表現します。

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

通常、ボックス化された型は必要ありません。ボックス化された型の使用は避け、代わりにプリミティブ型を使用してください。たとえば、String ではなく string を使用します。

TypeScript における共変性と反変性

共変性と反変性は、ジェネリック型の内部で型の関係がどう変わるかを表します。Dog は Animal ですが、すべての Animal が Dog とは限りません。

共変性（値を返す）：Dog を返す関数は、Animal を返す関数が必要な場所で使用できます。犬は動物ですが、任意の動物を返す関数は猫を返す可能性があります。

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // 有効
produceDog = produceAnimal; // エラー
```

反変性（値を受け取る）：どんな Animal も受け取れる関数は、Dog を受け取る関数が必要な場所で使用できます。犬にも対応できます

が、犬専用の関数ですべての動物を安全に処理することはできません。

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // 有効
consumeAnimal = consumeDog; // strictFunctionTypes が有効な場合はエラー
```

矢印は安全に代入できる方向を示します。

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (共変性)
- Consumer<Animal> -> Consumer<Dog> (反変性)

変更可能な配列: TypeScript では `Dog[]` を `Animal[]` に代入することも許可されます。両方の変数が同じ配列を参照するため、より広い型を通じて `Animal` を挿入すると、犬ではない値が犬の配列に入る可能性があります。

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // 許可されるが安全ではない
// dogs に Dog ではない Animal が含まれるようになる。
```

コンパイラオプション: `strictFunctionTypes` を有効にすると、独立した関数のパラメーターは反変的にチェックされます。無効の場合、安全でなくても両方向が許容されることがあります（双変性）。メソッドとコンストラクターのパラメーター宣言は、厳密な関数パラメーターのチェックの例外です。

型パラメーターのオプショナルな変性アノテーション

TypeScript 4.7 以降、out は共変性、in は反変性、in out は不変性を表します。通常は TypeScript が変性を推論します。アノテーションはジェネリック型パラメーターの使われ方と一致する必要があり、その振る舞いを任意に変えるものではありません。

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

テンプレート文字列パターンのインデックスシグネチャ

テンプレート文字列パターンのインデックスシグネチャを使用すると、テンプレート文字列パターンを使って柔軟なインデックスシグネチャを定義できます。この機能により、特定の文字列キーパターンでインデックス付けできるオブジェクトを作成でき、プロパティへのアクセスと操作をより細かく、明確に制御できます。

TypeScript はバージョン 4.4 以降、シンボルとテンプレート文字列パターンのインデックスシグネチャを許可しています。

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
```

```
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

satisfies 演算子

satisfies 演算子を使用すると、指定された型が特定のインターフェースや条件を満たしているかを確認できます。言い換えると、型が特定のインターフェースで必要とされるすべてのプロパティとメソッドを持っていることを保証します。変数が型の定義に適合することを保証する方法です。 例を示します。

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// `User` による型アノテーション
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// 以下の行では、TypeScript は適切に推論できない
user.attributes?.map(console.log); // プロパティ 'map' は型 'string | string[]' に存在しません。プロパティ 'map' は型 'string' に存在しません。
user.nickName; // string | string[] | undefined

// `as` による型アサーション
const user2 = {
    name: 'Simon',
```

```
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// ここでも、TypeScript は適切に推論できない
user2.attributes?.map(console.log); // プロパティ 'map' は型 'string | string[]' に存在しません。プロパティ 'map' は型 'string' に存在しません。
user2.nickName; // string | string[] | undefined

// `satisfies` 演算子を使うと、型を適切に推論できる
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log); // TypeScript は正しく推論する: string[]
user3.nickName; // TypeScript は正しく推論する: undefined
```

型のみのインポートとエクスポート

型のみのインポートとエクスポートを使用すると、その型に関連付けられた値や関数をインポートまたはエクスポートせずに、型をインポートまたはエクスポートできます。これはバンドルサイズを削減するのに役立つ場合があります。

型のみをインポートするには、`import type` キーワードを使用できます。

TypeScript では、`allowImportingTsExtensions` の設定に関係なく、型のみのインポートで宣言ファイルと実装ファイルの両方の拡張子（.ts、.mts、.cts、.tsx）を使用できます。

例：

```
import type { House } from './house.ts';
```

次の形式がサポートされています。

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

using 宣言と明示的なリソース管理

using 宣言は、破棄可能なリソースの管理に使用される、const に似たブロックスコープの不変バインディングです。値で初期化されると、その値の Symbol.dispose メソッドが記録され、その後、囲んでいるブロックスコープを抜ける際に実行されます。

これは ECMAScript のリソース管理機能に基づいています。この機能は、接続を閉じる、ファイルを削除する、メモリを解放するなど、オブジェクト作成後に不可欠なクリーンアップタスクを実行するのに役立ちます。

注意：

- TypeScript バージョン 5.2 で導入されてから日が浅いため、ほとんどのランタイムはネイティブにサポートしていません。Symbol.dispose 、 Symbol.asyncDispose 、 DisposableStack 、 AsyncDisposableStack、SuppressedError にはポリフィルが必要です。
- さらに、tsconfig.json を次のように設定する必要があります。

```
{
    "compilerOptions": {
        "target": "es2022",
```

```
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

例：

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // 簡単なポリフィル

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // リソースを宣言
    console.log(2);
} // リソースを破棄（例: `work[Symbol.dispose]()` を評価）

console.log(3);
```

コードの出力は次のようになります。

```
1
2
disposed
3
```

破棄の対象となるリソースは、Disposable インターフェースに準拠している必要があります。

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

using 宣言はリソースの破棄操作をスタックに記録し、宣言と逆の順序で破棄されることを保証します。

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // `C`、`B`、`A` の順に破棄。
```

後続のコードが実行された場合や例外が発生した場合でも、リソースは確実に破棄されます。そのため、破棄時に例外がスローされ、別の例外が抑制される可能性があります。抑制されたエラーの情報を保持するため、新しいネイティブ例外 SuppressedError が導入されています。

await using 宣言

await using 宣言は、非同期に破棄可能なリソースを処理します。値には Symbol.asyncDispose メソッドが必要で、ブロックの終了時に await されます。

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // リソースを宣言
} // リソースを破棄（例: `await work[Symbol.asyncDispose]()` を評価）
```

非同期に破棄可能なリソースは、Disposable または AsyncDisposable インターフェースのいずれかに準拠している必要があります。

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose ??= Symbol('Symbol.asyncDispose'); // 簡単なポリフィル

class DatabaseConnection implements AsyncDisposable {
    // オブジェクトが非同期に破棄されるときに呼び出されるメソッド
    [Symbol.asyncDispose]() {
        // 接続を閉じ、Promise を返す
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
        await new Promise(resolve => setTimeout(resolve, 1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
    // 新しい接続を作成し、スコープを抜けるときに非同期に破棄
    await using connection = new DatabaseConnection(); //  リソースを宣言
    console.log('Doing some work...');
} // リソースを破棄（例: `await connection[Symbol.asyncDispose]()` を評価）

doWork();
```

コードの出力は次のとおりです。

```
Doing some work...
Closing the connection...
Connection closed.
```

using 宣言と await using 宣言は、for、for-in、for-of、for-await-of、switch 文で使用できます。

インポート属性

TypeScript 5.3 のインポート属性（インポート用のラベル）は、モジュール（JSON など）の処理方法をランタイムに伝えます。これにより、インポートを明確にすることでセキュリティが向上し、より安全にリソースを読み込むためのコンテンツセキュリティポリシー（CSP）にも準拠します。TypeScript はそれらが有効であることを保証しますが、特定のモジュール処理における解釈はランタイムに委ねます。

例：

```
import config from './config.json' with { type: 'json' };
```

動的 import の場合：

```
const config = import('./config.json', { with: { type: 'json' } });
```

正規表現の構文チェック

TypeScript 5.5.4 以降、コンパイル時に正規表現リテラルの一般的なエラー（無効な構文、誤った後方参照、ターゲットの JS バージョンでサポートされていない機能など）がチェックされます。これはバグを早期に発見するのに役立ちますが、new RegExp(“…”) の文字列はチェックされません。

```
let r = /(a)\2/; // エラー: この後方参照は存在しないグループを参照しています。
```

import defer

`import defer` を使用すると、モジュールを読み込みつつ、そのモジュールから何かを実際に使用するまで実行を遅らせることができます。これにより、不要な処理や副作用を回避できます。

- 次の形式でのみ動作します：`import defer * as name from "module"`
- エクスポートにアクセスしたときにのみコードが実行されます